

Breaking Fault Lines: Unifying TEE-Assisted BFT Consensus in Partially Trusted Worlds

Anonymous Author(s)

Abstract

This paper revisits TEE-assisted BFT under a *universal partial-TEE model*, where an arbitrary subset of replicas execute inside TEEs while the remaining replicas operate without hardware trust guarantees. We show that heterogeneous trust changes the structure of quorum formation and fault tolerance. In particular, we derive a tight resilience bound $f < \max\{\frac{n}{3}, \frac{m}{2}\}$, where n is the total number of replicas and m is the number of TEE-enabled replicas. The result reveals a sharp threshold phenomenon: TEEs improve fault tolerance only once they exceed two-thirds of the deployment.

Guided by this characterization, we introduce two protocol principles: (1) a dual-quorum construction that safely combines TEE-only and mixed quorums, and (2) a TEE-leader fast path that leverages hardware-enforced non-equivocation to reduce both consensus and view-change latency. We realize these ideas in **RAFTEL**, which is, to our knowledge, the first HotStuff-style BFT protocol designed explicitly for arbitrary partial-TEE deployments, and in **CHAINED-RAFTEL**, a pipelined variant that further accelerates mixed-trust execution. We implement both protocols atop Intel SGX and evaluate them in LAN and WAN environments. Our results show that **RAFTEL** achieves up to 625 TPS with sub-670 ms latency in WAN settings, outperforming HotStuff by up to 308 TPS in throughput while approaching the performance of fully TEE-assisted protocols.

1 Introduction

Byzantine fault-tolerant (BFT) consensus protocols lie at the heart of distributed systems such as blockchains [24, 28]. They allow a network of replicas to agree on a sequence of transactions despite adversarial behaviors by a subset of replicas [7, 77]. This strong fault model provides robustness under adversarial behavior, but incurs substantial replication and communication overhead. To tolerate f Byzantine faults, classical BFT requires at least $3f + 1$ replicas—compared to $2f + 1$ for crash fault tolerance—and incurs heavy communication overhead, often involving three or more phases per decision [49]. These costs limit the scalability of BFT protocols in practice.

Trusted Execution Environments (TEEs), such as Intel SGX [31], offer a promising way to bridge this gap. By preventing equivocation through hardware guarantees, TEEs can reduce the quorum size (as low as $2f + 1$) and communication rounds of BFT protocols. In fact, prior TEE-assisted BFT protocols [15, 53, 76] have demonstrated near-CFT performance while preserving Byzantine resilience.

Unfortunately, most existing designs in the literature rely on a strong and increasingly unrealistic assumption: *all* replicas are equipped with TEEs. In practice, modern deployments are inherently *heterogeneous*. Cloud providers expose different trusted-computing technologies with varying capabilities and trust assumptions. TEE-enabled instances are often restricted to specific hardware types, regions, or pricing tiers. Large distributed systems evolve incrementally, making simultaneous migration of all replicas impractical. In permissionless and decentralized settings, requiring universal TEE provisioning further undermines openness and raises participation barriers.

Consequently, emerging systems increasingly operate in a *mixed-trust regime*, where only a subset of replicas can provide hardware-enforced guarantees. This setting fundamentally challenges existing consensus designs. Classical BFT assumes that no replicas are trusted, while prior TEE-assisted protocols assume homogeneous trust across all replicas. Real deployments satisfy neither assumption. Instead, consensus protocols must operate under *heterogeneous trust*, where TEE-enabled and non-TEE replicas coexist within the same protocol execution. This raises a fundamental question: *How should BFT consensus protocols exploit heterogeneous trust under arbitrary mixtures of TEE and non-TEE replicas?*

In this paper, we revisit TEE-assisted BFT under a *universal partial-TEE model* that generalizes both classical BFT [7, 77] and fully TEE-assisted consensus [15, 37, 53]. We consider a deployment of n replicas, among which only m replicas are TEE-enabled. Within this model, we derive a tight fault-tolerance bound:

$$f < \max\left\{\frac{n}{3}, \frac{m}{2}\right\},$$

where f denotes the number of Byzantine replicas.

This characterization reveals an important and previously overlooked phenomenon: partial TEE deployment does not improve resilience monotonically. Instead, the system exhibits a sharp threshold effect. When TEE-enabled replicas remain below two-thirds of the deployment (*i.e.*, $m/2 \leq n/3$), fault tolerance is still fundamentally bounded by classical BFT limits; TEEs can improve efficiency, but not resilience. Only once TEE-enabled replicas exceed this threshold (*i.e.* $m/2 > n/3$) does hardware-enforced non-equivocation directly increase fault tolerance.

Guided by this characterization, we identify two protocol principles for consensus under heterogeneous trust:

- **Dual-quorum construction.** Because TEE-enabled replicas cannot equivocate, safety can be established using either

a TEE-only quorum or a classical mixed quorum. We therefore introduce a dual-quorum design that safely combines: (i) a TEE-Quorum consisting exclusively of TEE votes, and (ii) a Mixed-Quorum consisting of arbitrary replicas. When sufficient TEE replicas are available, the protocol commits using smaller and faster TEE-Quorums; otherwise, it safely falls back to classical BFT quorum formation.

- **TEE-leader fast path.** Classical BFT protocols require additional communication phases to guard against leader equivocation during proposal and view-change. Under a TEE-enabled leader, however, equivocation is prevented by a TEE-protected trusted component, authenticated via hardware attestation [9, 36]. We exploit this property to bypass two phases and accelerate view synchronization, reducing both consensus latency and leader-change overhead while preserving compatibility with non-TEE execution paths.

We realize these ideas in **RAFTEL**¹, to the best of our knowledge, the first HotStuff-style BFT protocol designed explicitly for arbitrary partial-TEE deployments. **RAFTEL** builds on the chained structure and rapid leader rotation of HotStuff [77] and Damysus [15], while introducing heterogeneous quorum formation and TEE-aware fast paths. To support these mechanisms efficiently, we consolidate Damysus's trusted components into a unified trusted module, **CHECKER+**, which minimizes enclave transitions while supporting TEE-specific protocol logic.

We further propose **CHAINED-RAFTEL**, a pipelined variant that extends the chained execution model. In particular, **CHAINED-RAFTEL** introduces a *pipelined commit rule* in which blocks proposed by non-TEE leaders can be committed earlier once extended by TEE-led proposals. This optimization reduces latency even in mixed-trust deployments where only a subset of leaders are TEE-enabled.

We implement **RAFTEL** and **CHAINED-RAFTEL** atop Damysus using Intel SGX and evaluate them in both LAN and WAN environments. We compare against HotStuff, Damysus, and Achilles across deployments with up to $f = 32$ Byzantine faults. Our experimental results show that **RAFTEL** achieves up to 625 TPS with sub-670 ms latency in WAN settings. Compared with HotStuff, **RAFTEL** improves throughput by up to 1.02 $\times$ while reducing latency by 61%, approaching the performance of fully TEE-assisted protocols.

Contributions. The main contributions are as follows:

- **Universal partial-TEE model.** We formalize a universal model for TEE-assisted BFT consensus under arbitrary mixtures of TEE and non-TEE replicas, and derive a tight fault-tolerance bound characterizing consensus under heterogeneous trust.
- **Design principles for heterogeneous trust.** We introduce a dual-quorum construction and TEE-aware fast paths that

exploit hardware-enforced non-equivocation while remaining safe under arbitrary partial deployment.

- **Protocol design and implementation.** We design and implement **RAFTEL** and **CHAINED-RAFTEL**, the first HotStuff-style protocols supporting arbitrary partial-TEE deployments.
- **Evaluation.** We evaluate our prototype on Intel SGX in LAN and WAN settings, demonstrating substantial performance improvements over classical BFT protocols while approaching the performance of fully TEE-assisted systems.

2 Related Work and Motivation

2.1 Related work

We survey prior work on BFT consensus, focusing on TEE-assisted BFT protocols and modern Hybrid BFT protocols.

TEE-assisted BFT consensus. TEE-assisted protocols exploit hardware-enforced non-equivocation to reduce quorum sizes and communication phases. Early work, such as Hybster [3], uses trusted counters within TEEs to parallelize consensus instances, while FastBFT [45] leverages TEEs to implement secret sharing and achieve $O(n)$ communication complexity. Damysus [15] builds atop modern BFT designs such as HotStuff [77], enabling three-phase commit with linear communication complexity. Subsequent works, including OneShot [16], FlexiBFT [27], and Achilles [53], further reduce the number of communication phases or improve protocol efficiency. Another line of work executes the entire transaction processing pipeline inside TEEs to provide confidentiality guarantees [4, 57, 73]. Our work focuses on protocols that leverage the integrity guarantees of TEEs, although the theoretical insights derived here can also apply to confidentiality-oriented systems.

Hybrid fault models. Prior hybrid-fault models classify replicas according to failure semantics (e.g., crash versus Byzantine behavior). Early work [51, 68] introduced dual-failure models and proposed protocols tolerating bounded combinations of crash and Byzantine faults. UpRight [13] further showed that distinguishing crash faults from Byzantine faults can improve fault tolerance compared to traditional BFT models. Scrooge [60] explored mixed crash/Byzantine settings to reduce the cost of fast Byzantine replication in the presence of unresponsive replicas through replier quorums and message histories. Subsequent systems investigated additional hybrid assumptions. VFT [55], XFT [46], and RR [18] study orthogonal dimensions, including correlated failures, network synchrony, partition tolerance, and rollback attacks. However, these systems assume homogeneous replicas under a uniform fault model, where all replicas are treated identically and may potentially exhibit the same classes of faults. As a result, protocol design cannot exploit replica-specific identities or hardware-constrained behaviors to optimize consensus execution.

¹RAFTEL is a mythical island in One Piece [14], where all voyages converge and the ultimate truth uniting the world is said to reside.

SeeMoRe [1] adopts a mixed-trust deployment model in which private-cloud replicas are trusted (crash-only), while public-cloud replicas may behave Byzantine, and leverages trusted leaders to reduce communication phases. However, its optimization mainly relies on trusted leadership and does not fundamentally change quorum construction.

Hybrid TEE systems. Recent works have recognized the practicality of partial and incremental adoption [22, 39, 63]. ParTEETor [39] demonstrates that even limited TEE penetration in Tor can enhance resistance against deanonymization without degrading performance. In secure multiparty computation, LucidiTEE [63] shows that fairness exchange can be achieved if only t out of n participants are TEE-enabled (instead of all in [10]), while tolerating up to t ($t < n$) malicious parties. Inspired by this line of work, we study BFT consensus under partial TEE deployment; whereas prior efforts primarily target security, our work also emphasizes efficiency. Closer to our setting, Mixed Fault Tolerance (MFT) protocol [22] studies BFT consensus with partially TEE-enabled replicas, in which $n = 3f + 2$ replicas are required to tolerate f faults for leader election safety—a stricter bound than the conventional $3f + 1$ bound. However, MFT does not provide a systematic analysis of how the number of TEE-enabled replicas affects the fault-tolerance bound. Besides, MFT does not fully exploit the availability of TEEs to optimize protocol efficiency; in particular, it overlooks opportunities such as dual-quorum formation and fast commit rules.

2.2 Why a Universal Partial-TEE Model?

Existing TEE-assisted BFT protocols assume that every replica is equipped with a TEE. This assumption is often unrealistic in practice, where deployments are heterogeneous and only a subset of replicas may have access to TEE support.

- *TEE-enabled devices are not yet universally deployable.* TEEs are increasingly available across both cloud and edge platforms, including server-grade TEEs such as Intel SGX/TDX and AMD SEV-SNP, as well as Arm TrustZone on smartphones and mobile devices². However, TEE support is still limited to specific processor generations, server models, cloud instance types, or deployment configurations. Major cloud providers, including AWS, Alibaba Cloud, Microsoft Azure, and Google Cloud, provide limited TEE-enabled offerings. For example, AWS currently offers AMD SEV-SNP only on a few instance families (M6a, C6a, R6a) in select regions, often with additional cost overhead [61].
- *Incremental deployment is necessary.* Real-world systems evolve gradually. Migrating from non-TEE to fully TEE-enabled infrastructures takes time, requiring both hardware replacement and software redesign. This process naturally creates transitional stages where only a subset of replicas

are TEE-enabled, making partial deployments both common and realistic [39].

- *Hybrid infrastructure naturally arises in practical distributed deployments.* Many consortium, partially decentralized, and large-scale distributed systems operate across organizations and infrastructures with different hardware capabilities and operational constraints [6, 39]. In practice, some replicas may provision TEEs while others rely on commodity hardware due to differences in cost, deployment policies, hardware availability, or operational requirements. Consequently, these systems naturally form mixed-replica environments rather than uniformly TEE-enabled deployments.

These observations motivate a *universal* TEE-assisted BFT protocol that generalizes both classical BFT and fully TEE-assisted BFT, while capturing the mixed deployments common in practice.

3 System Model and Goals

3.1 System Model

We consider a system of n replicas, among which m replicas are equipped with TEEs and the remaining $n - m$ replicas execute without TEEs. We denote the set of TEE-enabled replicas by $\mathcal{S}_{\text{TEE}}$, where $|\mathcal{S}_{\text{TEE}}| = m$. We refer to replicas in $\mathcal{S}_{\text{TEE}}$ as *TEE replicas* and all others as *non-TEE replicas*. TEE replicas provide hardware-enforced execution integrity and non-equivocation guarantees, whereas non-TEE replicas may behave arbitrarily. We assume that each replica's TEE status remains fixed during a protocol execution; dynamic reconfiguration is discussed in Appendix F.

The model captures both classical BFT and fully TEE-assisted BFT as special cases. When $m = 0$, no replica has TEEs, and the setting reduces to classical BFT. When $m = n$, all replicas are TEE-enabled and the setting reduces to fully TEE-assisted BFT. Our focus is the general case $0 < m < n$, where replicas operate under heterogeneous trust.

Cryptographic setup. We assume a standard public-key infrastructure (PKI) for authenticating replicas and messages. Each replica p_i has a public/private key pair (pk_i, sk_i) . For TEE replicas, the signing (private) key used by the trusted component is generated and stored inside TEEs and cannot be extracted by the untrusted host. TEE replicas also support remote attestation [9, 36], allowing other replicas to verify both the identity of a TEE replica and the code executed inside its trusted component. Thus, all replicas can reliably determine whether a message was produced by a TEE-backed trusted component or by ordinary replica software.

Network model. We adopt the standard partially synchronous network model [20] used by many BFT protocols [7, 35, 77]. Before the unknown Global Stabilization Time (GST), messages may be delayed arbitrarily. After GST, there exists a known bound Δ such that every message sent between honest replicas is delivered within Δ . Replicas communicate

²Intel SGX has been deprecated on client-class processors [34].

over authenticated channels, and the adversary may delay, drop, reorder, or inject messages subject to cryptographic unforgeability.

Fault model. The system tolerates up to f Byzantine replicas, chosen from both TEE and non-TEE ones. Byzantine replicas may collude. For ease of presentation, we consider a Byzantine adversary that controls all Byzantine replicas. We distinguish three cases.

- **Honest replicas.** Honest replicas, whether TEE-enabled or not, follow the protocol.
- **Byzantine non-TEE replicas.** A corrupted non-TEE replica may equivocate, forge local state, send conflicting protocol messages, omit messages, or otherwise behave arbitrarily, subject only to standard cryptographic assumptions [7, 77].
- **Byzantine TEE replicas.** A corrupted TEE replica has an adversarial host operating system and untrusted application environment. The adversary may schedule, delay, replay, or reorder inputs and outputs to trusted components, and may invoke them with arbitrary inputs. However, it cannot extract secrets from the TEE, forge TEE-generated signatures, or cause the trusted component to execute code other than the attested protocol logic [15, 16, 21, 53].

TEE assumptions and non-goals. Our protocol relies on the integrity of the trusted component and the confidentiality of keys stored inside it. We do not attempt to defend against transient-execution attacks [8, 59, 71], micro-architectural side-channel attacks [62, 70, 72, 74, 75], or rollback/forking attacks [50, 52], as these are largely orthogonal to our protocol design. For example, rollback and forking attacks can be addressed using state-continuity mechanisms [23, 32, 50], which ensure monotonic evolution of trusted state. Such mechanisms may add local storage or cryptographic overhead, but do not change the quorum structure or communication pattern analyzed in this paper.

3.2 System Goals

Clients submit transactions to replicas, which order them into a sequence of blocks. Each block extends a parent block, forming a hash-linked chain. The protocol must satisfy:

- **Safety:** If two honest replicas commit two blocks b and b' at the same height, then $b = b'$.
- **Liveness:** After GST, every transaction submitted by an honest client is eventually included in a block committed by honest replicas, assuming the client retransmits to honest replicas as needed.

4 Pushing the Limits: Fault Tolerance Bound and Design Principles

We now characterize the resilience limits of consensus under heterogeneous trust and derive the protocol principles that guide RAFTEL. The key question is how the availability of

m TEE replicas (among n total replicas) affects both fault tolerance and efficient quorum formation.

4.1 Fault-Tolerance Bound

We first recall the two homogeneous extremes. In classical BFT, where no replica is trusted, consensus requires $n \geq 3f + 1$ replicas to tolerate f Byzantine faults [43]. That is, $f < n/3$ when $m = 0$. In fully TEE-assisted BFT, where all replicas are equipped with trusted components that prevent equivocation, consensus can tolerate up to a minority of Byzantine replicas, requiring only $n \geq 2f + 1$ replicas [12]. That is, $f < n/2$ when $m = n$.

The partial-TEE setting lies between these extremes but is not obtained by simply interpolating between them. We will show that consensus under heterogeneous trust has the following **tight resilience bound**:

$$f < \max\left\{\frac{n}{3}, \frac{m}{2}\right\}.$$

This bound says that, to tolerate f Byzantine faults, it suffices that either

$$n \geq 3f + 1 \quad \text{or} \quad m \geq 2f + 1.$$

The first condition corresponds to classical BFT: even without enough TEE replicas, safety can be maintained using mixed quorums over the full replica set. The second condition corresponds to a *TEE-dominated regime*: if sufficiently many replicas are TEE-enabled, their non-equivocation guarantees alone can support safe quorum formation.

This sharp transition is central to our design. It implies that TEEs should be used in two distinct ways: to form smaller quorums when enough TEE replicas are available, and to accelerate protocol phases whenever a TEE leader can prevent equivocation.

The proof for the above resilience bound extends the classical indistinguishability argument of Byzantine agreement [43] to the heterogeneous trust setting. By partitioning replicas into multiple groups and exploiting equivocation among non-TEE replicas, two honest groups can be forced to commit conflicting values, violating Safety. The full proof appears in Appendix B. The bound is also achievable, as demonstrated by the protocol presented later in this paper.

4.2 Principle 1: Dual-Quorum Construction

The resilience bound suggests that a protocol for heterogeneous trust should not rely on a single quorum rule. Instead, it should support two quorum types.

- **TEE-Quorum:** A TEE-Quorum consists of exclusively at least Q_T votes from TEE replicas, where $Q_T = \max\left\{\left\lfloor \frac{m}{2} \right\rfloor + 1, f + 1\right\}$. The majority term ensures that any two TEE-Quorums intersect in at least one TEE replica. The $f + 1$ term ensures that the quorum contains at least one honest TEE replica. Since TEE-backed votes are non-equivocating even when the host is Byzantine, the intersection is sufficient to prevent conflicting certificates.

- **Mixed-Quorum:** A Mixed-Quorum consists of at least Q_M votes from arbitrary replicas, where $Q_M = n - f$. This is the standard BFT quorum size. Any two Mixed-Quorums intersect in at least $n - 2f$ replicas, which contains an honest replica when $n \geq 3f + 1$.

Safety across quorum types. The key requirement is not merely that each quorum type is safe in isolation, but that different quorum types remain safe when used interchangeably. Under the bound above, any two valid quorums—TEE/TEE, mixed/mixed, or TEE/mixed—intersect in at least one replica that cannot equivocate. This property allows the protocol to form certificates using whichever quorum becomes available first, while preserving a single global safety argument.

This dual-quorum construction lets the protocol adapt to heterogeneous deployments. When enough TEE replicas respond quickly, the leader can form a smaller TEE certificate. When TEE replicas are slow, unavailable, or insufficient, the protocol falls back to a classical Mixed-Quorum without changing the safety rule. The formal quorum construction and correctness proof are presented in Section 5 and Appendix C.1, respectively.

4.3 Principle 2: TEE-Leader Fast Path

The second design principle exploits a different consequence of trusted hardware: a TEE-enabled leader cannot equivocate.

1) *TEE-leader fast path.* Classical BFT protocols require additional phases to protect against a Byzantine leader that proposes conflicting blocks in the same view. For example, prepare-style phases ensure that replicas observe a consistent proposal before committing. By contrast, a TEE-enabled leader can produce at most one valid proposal for a given view, and the proposal is bound to the attested protocol logic. Thus, the protocol can safely bypass phases whose sole purpose is to defend against leader equivocation. In RAFTEL, this yields a fast path for TEE leaders: consensus can complete with fewer communication phases, while non-TEE leaders use the standard multi-phase path. Thus, the protocol remains safe under arbitrary leader schedules but obtains lower latency whenever the current leader is TEE-enabled.

2) *Fast view-change.* TEE-based non-equivocation also simplifies view change. In classical BFT, a new leader typically collects new-view messages or the highest prepared certificate to determine which block is safe to extend, because the previous leader may have equivocated. If the previous leader was TEE-enabled, however, there can be at most one valid proposal from that view.

Therefore, the next leader can safely inherit the unique TEE-certified proposal rather than reconstructing safety solely from a full set of new-view messages. This reduces leader-change latency and improves responsiveness under frequent leader rotation. Importantly, this optimization is conditional: when the previous leader is not TEE-enabled, the protocol falls back to the standard view-change rule.

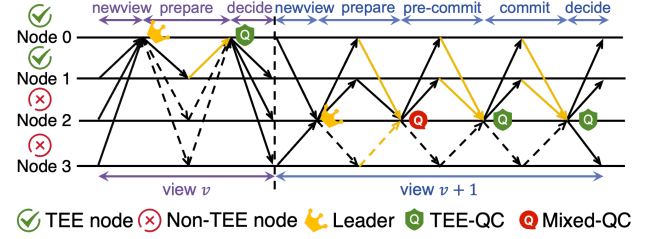


Figure 1. RAFTEL Overview. Solid arrows denote prioritized broadcasts to TEE replicas, while dashed arrows denote subsequent broadcasts to non-TEE replicas. Yellow arrows highlight the messages that arrive at the leader first.

4.4 Implications

Together, these principles translate the fault-tolerance characterization into protocol structure. The dual-quorum construction exploits heterogeneous trust at the quorum level, while the TEE-leader fast path and fast view change exploit non-equivocation at the leader level. This separation is important: even when TEEs are too sparse to improve resilience, they can still improve performance by reducing quorum size, shortening the commit path, or accelerating leader transitions. The next sections instantiate these principles in RAFTEL and its pipelined variant, CHAINED-RAFTEL.

5 RAFTEL Design

5.1 Overview

RAFTEL is a mixed-trust TEE-assisted BFT protocol that operates under arbitrary mixtures of TEE-enabled and non-TEE replicas. Rather than relying on a binary fallback between a TEE-based protocol and a non-TEE protocol, RAFTEL is designed as a unified protocol for mixed-replica settings that can continuously adapt to different levels of TEE availability.

At a high level, all replicas follow the same protocol structure, but differ in how they handle security-critical steps. TEE replicas invoke a trusted component called CHECKER+ on critical paths to generate certified messages and enforce non-equivocation constraints across protocol steps. Replicas without TEEs execute these steps entirely outside trusted hardware, without the additional guarantees provided by CHECKER+. As illustrated in Fig. 1, RAFTEL combines two key ideas introduced in the previous section: (1) a dual-quorum, and (2) TEE-leader fast path.

The dual-quorum design (§4.2) enables RAFTEL to dynamically switch between two quorum formation modes. When sufficient TEE-backed votes are available, the protocol forms a TEE-Quorum using only TEE replicas, allowing smaller quorum certificates and faster progress. Otherwise, the protocol safely falls back to a Mixed-Quorum formed from arbitrary replicas, preserving compatibility with classical BFT quorum formation under mixed-trust deployments.

Hybrid trust also enables leader-dependent commit paths (§4.3). When the leader is TEE-enabled, the trusted component constrains certified proposals generated by the leader, substantially reducing the coordination overhead required to tolerate equivocation. This allows the protocol to shorten the critical commit path and accelerate leader transitions. When the leader is not TEE-enabled, RAFTEL safely falls back to the standard BFT commit path.

Pacemaker. RAFTEL adopts a standard partially synchronous pacemaker similar to those used in HotStuff-style BFT protocols [69, 77]. The pacemaker is responsible for ensuring liveness after GST, while safety is guaranteed independently through quorum intersection and TEE-enforced non-equivocation. Our protocol design is orthogonal to the pacemaker and does not modify its underlying mechanism.

5.2 Definitions

Views, phases, and steps. RAFTEL proceeds in views, each coordinated by a unique leader known to all replicas. We denote the leader of view v by $\text{ldr}(v)$. Replicas may be either TEE-enabled or non-TEE replicas. We use $\text{isTEE}(id)$ to denote whether replica id is TEE-enabled, which can be verified through remote attestation [36]. Each view consists of multiple phases identified by a phase tag $ph \in \{\text{nv}, \text{prep}, \text{prec}, \text{com}\}$, corresponding to new-view, prepare, pre-commit, and commit, respectively. We define a *step* as a pair (v, ph) representing phase ph in view v .

For a non-TEE leader, a view progresses sequentially through all phases: $(v, \text{nv}) \rightarrow (v, \text{prep}) \rightarrow (v, \text{prec}) \rightarrow (v, \text{com}) \rightarrow (v+1, \text{nv})$. If the leader of view v is a TEE replica, the protocol skips the *prec* and *com* phases, and the progression becomes: $(v, \text{nv}) \rightarrow (v, \text{prep}) \rightarrow (v+1, \text{nv})$. We use $(v, ph)++$ to denote the transition to the next phase.

Blocks and chains. A block contains client transactions and the hash of its parent block. Blocks are linked through hash references to form a chain rooted at the genesis block $\mathcal{G}$. We write $b_1 > b_2$ if block b_1 extends block b_2 . Similarly, $b_1 > h$ denotes that b_1 extends the block with hash value h . Two blocks conflict if neither extends the other. The height of a block is its distance from the genesis block. We compare block freshness using their associated views, where blocks proposed in higher views are considered more recent. We assume a function $\text{createLeaf}(h_p)$ that creates a new block extending the parent block with hash h_p .

Certificates. Replicas authenticate protocol messages using digital signatures. We use $\langle msg \rangle_\sigma$ to denote a signed message carrying signature σ , and $\langle msg \rangle_{\vec{\sigma}}$ to denote a message carrying a set of signatures $\vec{\sigma}$.

A *certificate* is a signed statement of the form $\phi = \langle h, v, h', v', ph \rangle_{\vec{\sigma}}$, where h and v denote the hash and view of the certified block, h' and v' denote the hash and view of the justified block (i.e., block can be safely extended), and ph denotes the

phase. Given a certificate ϕ of the form $\langle h, v, h', v', ph \rangle_{\vec{\sigma}}$, let $\phi.H_{prep}$ be h ; $\phi.V_{prep}$ be v ; $\phi.H_{just}$ be h' ; $\phi.V_{just}$ be v' ; and $\phi.sign$ be $\vec{\sigma}$. We use $\vec{\phi}$ to denote a list of certificates, and $\vec{\phi}^n$ to indicate that the list has length n . We use $\perp$ for unused fields when a value is not required. Certificates may be generated either by the trusted component of a TEE-enabled replica or by the local software logic of a non-TEE replica.

Quorum certificates. A *Quorum Certificate* (QC) is an aggregated proof that a quorum of replicas voted for the same block in the same view and phase. Following Damysus [15], QCs are constructed by aggregating partial certificates using multi-signatures: $\text{Combine}([\langle h, v, h', v', ph \rangle_{\sigma_1}, \dots, \langle h, v, h', v', ph \rangle_{\sigma_n}]) = \langle h, v, h', v', ph \rangle_{\{\sigma_1, \dots, \sigma_n\}}$.

RAFTEL supports two types of quorum certificates corresponding to the dual-quorum design:

- **TEE-QC:** $qc \leftarrow \text{Combine}(\vec{\phi}^{Q_T})$, a list of Q_T certificates issued exclusively by TEE replicas.
- **Mixed-QC:** $qc \leftarrow \text{Combine}(\vec{\phi}^{Q_M})$, a list of Q_M certificates from any combination of TEE and non-TEE replicas.

Given a list of certificates $\vec{\phi} = [\phi_1, \dots, \phi_n]$, we define the following operations to check whether *quorum* certificates have been received: let $\text{T-match}(\vec{\phi}, k, h, v, ph)$ be true iff: (1) $n = k$; (2) all n signatures have been created by different TEE replicas; and (3) $\forall i \in 1, \dots, n, h = h_i \wedge v = v_i \wedge ph = ph_i$. Similarly, $\text{M-match}(\vec{\phi}, k, h, v, ph)$ is defined with the same conditions except that in (2) the n signatures may come from different *arbitrary replicas* (TEE or non-TEE).

5.3 Trusted Components

In RAFTEL, TEE replicas invoke trusted logic inside TEEs through a trusted component called CHECKER+. The role of CHECKER+ is to maintain trusted protocol state and enforce consistency constraints across certified protocol steps. In particular, CHECKER+ binds certified messages to monotonically increasing protocol identifiers and prevents conflicting certified protocol states from being generated by the same TEE-enabled replica.

RAFTEL builds upon the trusted components originally used in Damysus, namely CHECKER and ACCUMULATOR (§D.2). At a high level, CHECKER validates proposal safety and binds certificates to monotonically increasing $(view, phase)$ identifiers, while ACCUMULATOR tracks the latest prepared state carried in leader-transition messages. In RAFTEL, these functionalities are tightly coupled in the protocol execution path. Therefore, we consolidate them into a single trusted module, called CHECKER+, and adapt it to support the dual-quorum construction and leader-dependent fast paths. This consolidation reduces redundant enclave invocations and minimizes context switches between trusted and untrusted execution.

CHECKER+ maintains two trusted protocol states: the latest prepared block and the latest locked block. The locked block (i.e., the highest block for which the replica holds a valid QC)

Algorithm 1 The pseudocode of operations for replica i in RAFTEL

```

1:  $pks$  // public keys
2:  $view = 0$  // current view
3:  $qc_{prep}$  // last prepared certificate
4:
5: // prepare phase
6: as a leader
7:   waits for  $\vec{\phi}$  s.t. T-match ( $\vec{\phi}, Q_T, \perp, view, nv$ )
       $\vee$  M-match ( $\vec{\phi}, Q_M, \perp, view, nv$ )
8:    $\phi_{nv} :=$  certificate  $\phi \in \vec{\phi}$  with highest  $\phi.V_{just}$ 
9:    $b := \text{createLeaf}(\phi_{nv}.H_{just}, txs)$ 
10:  if isTEE( $i$ ) then
11:     $\phi := \text{TEEprepare}(b, H(b), \phi_{nv}, \vec{\phi})$ 
12:    send  $\langle \phi_{nv}, b, \phi \rangle$  to all
13:  else
14:    PrioBroadcast ( $\langle \phi', b, \langle H(b), view, prep \rangle_{\sigma'} \rangle$ )
15:  endif
16: all replicas
17:   waits for  $\langle \langle h', v', nv \rangle_{\sigma}, view, b, prep \rangle_{\sigma'}$  from the leader
18:    $\phi_{nv} := \langle h', v', nv \rangle_{\sigma}$ 
19:    $\phi_{prep} := \langle H(b), view, h', v', prep \rangle_{\sigma'}$ 
20:   if isTEE( $i$ ) then
21:      $\phi' := \text{TEEprepare}(H(b), \phi_{prep}, \phi_{nv})$ 
22:   else
23:     abort if  $\neg(\text{VERIFY}(\phi_{prep}) \wedge b > h')$ 
24:     if isTEE( $\text{ldr}(view)$ ) then
25:        $\phi' := \langle H(b), view, com \rangle_{\sigma_i}$ 
26:     endif
27:     else  $\phi' := \langle H(b), view, prep \rangle_{\sigma_i}$ 
28:     endif
29:   endif
30:   send  $\phi'$  to leader
31:
32: // pre-commit phase
33: as a leader
34:   waits for  $\vec{\phi}$  s.t. T-match ( $\vec{\phi}, Q_T, \perp, view, prep$ )
       $\vee$  M-match ( $\vec{\phi}, Q_M, \perp, view, prep$ )
35:   PrioBroadcast ( $qc_{prep} := \text{Combine}(\vec{\phi})$ )
36: all replicas
37:   waits for  $\langle h, view, \perp, prep \rangle_{\vec{\phi}}$  from the leader
38:   abort if  $\neg(\text{VERIFY}(\langle h, view, \perp, prep \rangle_{\vec{\phi}}))$ 
39:   if isTEE( $i$ ) then  $\phi' := \text{TEEstore}(H(b), \phi_{prep})$ 
40:   else  $\phi' := \langle H(b), view, prec \rangle_{\sigma_i}$ 
41: endif
42:   send  $\phi'$  to leader
43:
44: // commit phase
45: as a leader
46:   waits for  $\vec{\phi}$  s.t. T-match ( $\vec{\phi}, Q_T, \perp, view, prec$ )
       $\vee$  M-match ( $\vec{\phi}, Q_M, \perp, view, prec$ )
47:   PrioBroadcast ( $qc_{prec} := \text{Combine}(\vec{\phi})$ )
48: all replicas
49:   waits for  $\langle h, view, \perp, prec \rangle_{\vec{\phi}}$  from the leader
50:   abort if  $\neg(\text{VERIFY}(\langle h, view, prec \rangle_{\vec{\phi}}))$ 
51:   if isTEE then  $\phi' := \text{TEEstore}(H(b), \phi_{prec})$ 
52:   else  $\phi' := \langle H(b), view, com \rangle_{\sigma_i}$ 
53:   endif
54:   send  $\phi'$  to leader
55:
56: // decide phase
57: as a leader
58:   waits for  $\vec{\phi}$  s.t. T-match ( $\vec{\phi}, Q_T, \perp, view, com$ )
       $\vee$  M-match ( $\vec{\phi}, Q_M, \perp, view, com$ )
59:   PrioBroadcast ( $qc_{com} := \text{Combine}(\vec{\phi})$ )
60: all replicas
61:   waits for  $\langle h, view, \perp, com \rangle_{\vec{\phi}}$  from the leader
62:   abort if  $\neg(\text{VERIFY}(\langle h, view, \perp, com \rangle_{\vec{\phi}}))$ 
63:   execute  $b$  corresponding to  $h$  and reply to clients
64:
65: // new-view phase
66: upon timeout
67:    $view++$ 
68:   if isTEE( $i$ ) then  $\phi_{nv} = \text{TEEview}()$ 
69:   else  $\phi_{nv} := \langle nv, view, qc_{prep} \rangle_{\sigma_i}$ 
70:   endif
71:   send  $\phi_{nv}$  to  $view$ 's leader
72:
73: function PrioBroadcast (msg)
74:   if  $|S_{TEE}| \geq Q_T$  then
75:     select  $S_{prio} \subseteq S_{TEE}$ 
76:     send msg to  $S_{prio}$ 
77:     send msg to all other replicas
78:   else
79:     send msg to all
80:   endif

```

preserves safety by preventing conflicting executions, while the prepared block supports liveness by allowing replicas to relay their latest safe state during leader transitions. By integrating the functionality of ACCUMULATOR, CHECKER+ also enables a TEE-enabled leader to safely select and extend the highest prepared block carried in leader-transition messages.

CHECKER+ state. The state maintained by replica p_i 's CHECKER+ consists of three components:

- $\{sk_i, pk_1, \dots, pk_n\}$, where sk_i is the enclave-protected private key of replica p_i , and $\{pk_1, \dots, pk_n\}$ are public keys;
- $(view, phase)$, where $view$ is the current protocol view and $phase$ is the current protocol phase. This identifier monotonically increases whenever CHECKER+ generates a certified protocol message;

- $(prep_v, prep_h)$ and $(lock_v, lock_h)$, which record the view number and hash of the latest prepared block and latest locked block, respectively.

CHECKER+ interface. The CHECKER+ component exposes three trusted operations:

- **TEEprepare** $(b, h, \phi_{nv}, \vec{\phi})$: takes a proposed block b with hash h , a highest-view leader-transition certificate ϕ_{nv} , and a set of leader-transition certificates $\vec{\phi}$. It verifies that $\vec{\phi}$ satisfies the quorum rule and that b safely extends the prepared block certified by ϕ_{nv} . The function then generates a certified proposal tagged with the current $(view, phase)$ and increments the local protocol identifier.
- **TEEstore** (ϕ) : takes a certified protocol message ϕ , verifies its validity, and updates the trusted prepared or locked state according to the protocol step associated with ϕ . It then outputs a certified confirmation of the state update.
- **TEEsingn** (ϕ) : generates a certified message for the currently stored prepared state, which is used during leader transitions and quorum formation.

5.4 The Algorithm

Algorithm 1 and Algorithm 2 present the pseudocode of replica operations, where Algorithm 2 is executed only by TEE replicas. At a high level, RAFTEL follows two execution paths depending on the leader type. Under a TEE-enabled leader, the protocol leverages trusted state and certified proposal consistency enforced by CHECKER+ to shorten the commit path. Under a non-TEE leader, the protocol falls back to a standard BFT-style quorum path while preserving safety through quorum intersection.

View change. When entering a new view, replicas send new-view certificates carrying their latest prepared state to the new leader (Algorithm 1, lines 66–71). The leader collects a valid quorum of certificates and selects the highest prepared block as the safe extension point for the new proposal.

TEE-leader fast path. When the leader is TEE-enabled, it invokes **TEEprepare** to generate a certified proposal extending the highest prepared block carried in the new-view certificates. The trusted component binds the proposal to the current protocol step (Algorithm 2, line 18) and prevents conflicting certified proposals from being generated by the same TEE-enabled leader.

The certified proposal $\langle \phi', b, \langle H(b), view, prep \rangle_\sigma \rangle$ is then broadcast to replicas. Upon receiving the proposal, replicas verify that the proposed block safely extends the justified block carried in the certificate. TEE replicas invoke **TEEprepare** to generate certified responses, while non-TEE replicas return signed prepare certificates (Algorithm 1, lines 20–25). Since certified proposals generated by the TEE-enabled leader are uniquely constrained by CHECKER+, the protocol can safely use a shortened commit path under this execution mode. Thus, backups directly generate commit certificates

Algorithm 2 TEE code for operations

```

1:  $(sk, pks)$  // private and public key
2:  $(view, phase) = (0, 0)$  // current view and phase
3:  $(prep_v, prep_h) = (0, H(\mathcal{G}))$  // latest prepared block
4:  $(lock_v, lock_h) = (0, H(\mathcal{G}))$  // latest locked block
5:
6: function TEEsingn  $(h, h', v')$ 
7:    $\phi := \langle h, view, h', v', phase \rangle_\sigma$ 
8:    $(view, phase) ++$  // increased to avoid equivocation
9:   return  $\phi := \langle h, view, h', v', phase \rangle_\sigma$ 
10:
11: function TEEprepare  $(b, h, \phi_{nv}, \vec{\phi})$ 
12:   if  $\left( \begin{array}{l} (|\{\phi \in \vec{\phi}_n : (\phi.signer) \in S_{TEE}\}| \geq Q_T) \\ \vee (|\vec{\phi}_n| \geq Q_M) \wedge \phi_n \in \vec{\phi}_n \wedge \\ (\forall \phi' \in \vec{\phi}_n \text{ where } \phi' \equiv \langle \tilde{h}, \tilde{v}, \tilde{v}' \rangle \wedge \tilde{v}' = view \\ \wedge v \geq \tilde{v})) \end{array} \right)$ 
13:   then
14:      $\langle \perp, v, h', v', ph \rangle_\sigma := \phi_{nv}$ 
15:     abort if  $\neg(\text{VERIFY}(\sigma) \wedge v = view \wedge ph = nv)$ 
16:     abort if  $\neg(H(b) = h \wedge b.h_p = h')$ 
17:     abort if  $\neg(h' = lock_h \vee v' > lock_v)$ 
18:     if isTEE  $(ldr(view)) \wedge ldr(view) \neq i$  then
19:        $prep_v = v; prep_h = h$ 
20:     endif
21:     return  $\phi := \text{TEEsingn}(h, h', v')$ 
22:
23: function TEEstore  $(\phi)$ 
24:   abort if  $\neg(\text{VERIFY}(\phi)_{pks} \wedge v \geq view)$ 
25:    $prep_v = v; prep_h = h$ 
26:   if  $ph = prec$  then  $lock_v = v; lock_h = h$ 
27:   return  $\phi := \text{TEEsingn}(h, \perp)$ 
28:
29: function TEEview  $()$ 
30:   return  $\phi := \text{TEEsingn}(\perp, prep_h, prep_v)$ 

```

without proceeding through the intermediate *pre-commit* and *commit* phases used under non-TEE leaders (Algorithm 1, line 25). Once the leader collects a valid *quorum* of commit certificates, it combines them into a commit QC and broadcasts the QC to all replicas. Upon receiving the commit QC, replicas execute the block.

Non-TEE leader fallback path. When the leader is not TEE-enabled, RAFTEL follows a standard HotStuff-style commit path. The leader first broadcasts a proposal extending the highest prepared block collected during the leader transition. Replicas validate the proposal and generate prepare responses according to their trust configuration. TEE replicas invoke **TEEprepare** to generate certified prepare responses, while non-TEE replicas return signed prepare certificates (Algorithm 1, lines 20–23, 27). The remaining protocol phases

follow the same quorum-collection pattern. The leader collects a valid quorum of certificates, combines them into a quorum certificate, and broadcasts the QC to replicas. TEE replicas update their trusted prepared or locked states through **TEEstore**, while non-TEE replicas generate signed protocol votes locally (Algorithm 2, lines 24, 25). This process repeats across the *prepare*, *pre-commit*, and *commit* phases until the leader gathers a valid commit quorum certificate and broadcasts the final commit certificate for execution.

Dual-quorum processing. Throughout the protocol, quorum formation follows the dual-quorum construction introduced in §4.2. A valid quorum certificate may therefore be formed either from a TEE-Quorum consisting exclusively of TEE replicas or from a Mixed-Quorum consisting of arbitrary replicas (Algorithm 1, lines 7, 34, 46, and 58).

When sufficient TEE replicas are available, leaders prioritize collecting TEE-backed certificates to accelerate quorum formation. To maximize the likelihood of forming a TEE-Quorum, leaders invoke **PrioBroadcast** (msg), which opportunistically disseminates proposals to TEE replicas before extending the broadcast to all replicas.

5.5 Correctness Analysis

We provide a sketch of **RAFTel**'s safety here, while leaving the full proofs of safety and liveness in Appendix C.1.

Safety. We show that no two honest replicas execute conflicting blocks. Under a TEE leader, non-equivocation ensures that only one block is proposed per view, while under a non-TEE leader, quorum intersection guarantees that at least one non-equivocating replica carries forward the latest prepared block; therefore, every prepared block extends previously prepared blocks, ensuring safety.

6 CHAINED-RAFTel Design

6.1 Overview

CHAINED-RAFTel extends **RAFTel** with a chained pipeline structure inspired by Chained-HotStuff [77] and Damysus [15]. Instead of completing all protocol phases for a block within a single view, **CHAINED-RAFTel** pipelines justification and commitment across consecutive views. As a result, leaders of adjacent views overlap in responsibility: each leader both proposes a new block and helps justify or commit blocks proposed in earlier views.

At a high level, blocks carry quorum certificates that justify their parent blocks. When a leader proposes a new block, the QC embedded in the proposal simultaneously certifies the predecessor block. This pipelined structure allows commitment decisions to propagate continuously along the chain while reducing coordination overhead across views.

Most importantly, hybrid trust introduces asymmetric commit depth in the chained pipeline. Blocks proposed by TEE-enabled leaders can be committed after forming a one-chain, while blocks proposed by non-TEE leaders require a

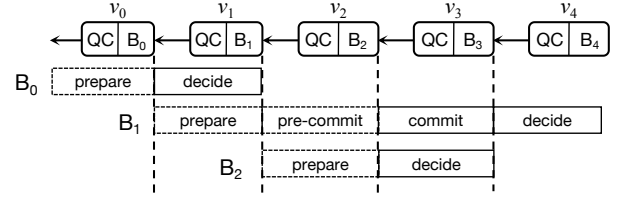


Figure 2. CHAINED-RAFTel allows replicas to pre-commit the predecessor block b_2 of a block b_3 when preparing b_3 , while simultaneously committing b_2 's predecessor block b_1 .

three-chain to preserve safety. As a result, the commit latency of a block depends on the trust guarantees of the leader who proposed it.

Fast commit. The chained structure also enables cross-view acceleration. When a non-TEE leader's block is immediately followed by a block proposed by a TEE-enabled leader, the TEE-backed proposal can safely accelerate commitment of its predecessor. For example, in Fig. 2, when block b_2 is committed, its predecessor b_1 is simultaneously committed without waiting for the commit phase of the next view. Consequently, some non-TEE blocks may also be finalized earlier under mixed-trust leader sequences. If a leader fails to gather sufficient votes, replicas fall back to standard new-view processing. Replicas send new-view messages carrying their latest prepared certificates, allowing the next leader to safely reconstruct the highest prepared chain and continue progress.

6.2 Definitions

We now detail how we adapt some of the concepts introduced previously to handle a chained version.

Blocks and quorum certificates. As in Chained-HotStuff [77], blocks are organized into a chained structure. We replace **createLeaf** with **createChain**, which generates chained blocks while filling missing gaps in the chain if necessary. Each block stores: (1) a hash pointer to its parent block, denoted by $b.parent$; and (2) a quorum certificate stored in $b.just$, which justifies the parent block.

A quorum certificate has the form $qc = \langle v, h \rangle_{\vec{\sigma}}$, where (v, h) identify the justified block and $\vec{\sigma}$ is the set of partial signatures collected for that block. As in **RAFTel**, quorum formation follows the dual-quorum construction and may therefore use either TEE-Quorum or Mixed-Quorum certificates.

Pipeline structure. The chained protocol contains only two protocol phases: *prepare* and *new view*, identified by tags **prep** and **nv**, respectively. Leaders continuously pipeline proposals across views, allowing justification and commitment to overlap between consecutive blocks. A block forms a *one-chain* if it directly extends the block referenced by its QC. Similarly, a block forms a *three-chain* if three consecutive parent-linked blocks carry consecutive justifications.

6.3 Commit Semantics

The commit rule in CHAINED-RAFTEL depends on both the chain structure and the trust guarantees of the leader.

TEE-enabled leaders. If a block proposed by a TEE-enabled leader forms a one-chain, the block and all of its ancestors are committed immediately. This optimization is enabled by the trusted proposal consistency enforced by CHECKER+, which prevents conflicting certified proposals from being generated for the same protocol step.

Non-TEE leaders. If a block proposed by a non-TEE leader forms a three-chain, the head of the three-chain and all of its ancestors are committed. This rule follows the standard chained BFT commit logic and preserves safety through quorum intersection across consecutive views.

Mixed-trust acceleration. The chained pipeline also enables mixed-trust commit acceleration. When a non-TEE leader's block is immediately followed by a block proposed by a TEE-enabled leader, the TEE-backed proposal may safely accelerate commitment of its predecessor. As a result, some non-TEE blocks can be finalized earlier than under the standard three-chain rule without compromising safety.

6.4 The Algorithm

Algorithm 3 in Appendix G presents the pseudocode of CHAINED-RAFTEL. The protocol follows a pipelined *prepare-new view* workflow in which consecutive leaders overlap in both proposal generation and predecessor justification.

Prepare phase. At the beginning of each view, the leader constructs a new chained block extending the highest prepared certificate currently known to the replica. If the leader already holds the latest prepared QC from the previous view, it directly extends that chain. Otherwise, it reconstructs the latest prepared chain from new-view messages.

The leader then broadcasts the proposal together with its justification certificate. TEE-enabled leaders invoke **TEEprepare** to generate certified proposals, while non-TEE leaders attach signed prepare messages. Replicas validate the proposal chain and return prepare responses according to their trust configuration.

New-view phase. At the end of a view, replicas forward their latest prepared certificates to the next leader. If the current leader fails to gather a valid quorum, replicas increment their local views and continue protocol execution using the highest prepared certificate carried in new-view messages.

Pipelined commitment. Unlike RAFTEL, commitment decisions are not completed within a single view. Instead, quorum certificates carried by newly proposed blocks continuously justify and finalize predecessor blocks along the chain. As the chain grows, replicas evaluate the one-chain or three-chain commit conditions according to the trust guarantees of the corresponding leaders.

6.5 Correctness Analysis

The proof of CHAINED-RAFTEL's safety and liveness properties is similar to RAFTEL's except for the difference caused by the pipelining structure. Due to space constraints, we leave the detailed proof of safety and liveness in Appendix C.2.

7 Performance Evaluation

We implement a prototype of RAFTEL and CHAINED-RAFTEL³, and evaluate their performance in both LAN and WAN. We compare RAFTEL and CHAINED-RAFTEL with HotStuff, Basic-Damysus, and Achilles to show the performance improvements. We aim to answer the following questions:

- **Q1:** How does RAFTEL perform with varying replicas in WAN and LAN compared to its counterparts?
- **Q2:** How does RAFTEL perform under different cases?
- **Q3:** How much performance overhead is introduced by SGX-related operations?

7.1 System Implementation and Setup

Implementation. We use Intel SGX to provide trusted services and develop atop the Damysus implementation⁴. All protocols are implemented in C++. We build on Damysus's trusted components, CHECKER and ACCUMULATOR, and consolidate their functionalities into a unified trusted module, CHECKER+, which is adapted to support our dual-quorum construction and fast-path designs. The detailed design is presented in §5.3. We use the OpenSSL library [33] to realize ECDSA signatures with *prime256v1* elliptic curves and Salticidae [17] for replica connections. Following the prior work [15, 53], we do not implement remote attestation in the prototype, which is needed during initialization and is outside the steady-state performance path.

Experimental setup. We conduct all experiments on a public cloud platform using up to 97 SGX-enabled instances, with each replica deployed on a dedicated virtual machine. Each VM is provisioned with 8 vCPUs, 32 GB of RAM, and runs Ubuntu Linux 20.04. All instances are connected through a private network interface with a bandwidth of 10 Gbps. We evaluate two deployment scenarios: a local area network (LAN) and a wide area network (WAN). In the LAN setting, the average inter-replica RTT is 0.1 ± 0.02 ms. Because SGX-enabled instances are only available in a limited set of regions, we emulated WAN conditions using NetEm [29], configuring the inter-replica RTT to 100 ± 5 ms.

Metrics. We consider two types of performance metrics: (1) commit throughput measures the number of committed transactions per second (TPS), while commit latency measures the average delay from when a leader proposes transactions to when they are executed; (2) end-to-end throughput

³Code Availability: <https://github.com/Artifact2026/Raftel>.

⁴Available at <https://github.com/vrahli/damysus>.

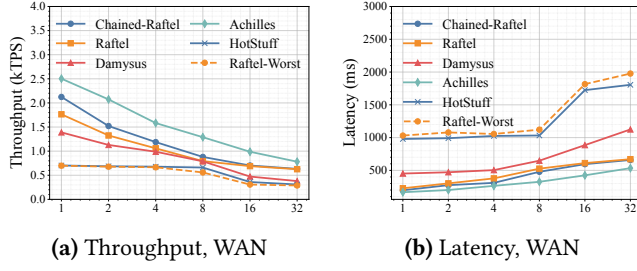


Figure 3. Throughput and latency of RAFTEL with varying faults in WAN.

measures the number of client replies received per second (TPS), while end-to-end latency measures the average delay from when clients create transactions to when replies are received. In most experiments, following Damysus, we use commit throughput/latency, since these metrics reduce the impact of client-side overheads and enable fairer protocol-level comparisons. We only use end-to-end throughput/latency in Fig. 6 and Fig. 10 to evaluate overall system scalability and client-perceived performance.

7.2 Scalability Evaluation

We evaluate the performance of RAFTEL and CHAINED-RAFTEL in the best case, *i.e.*, when the number of TEE replicas is larger than f to enable dual-quorum, and the leader is TEE-enabled to support the fast path, with the system parameters set to $m = f + 1$ and $n = 3f + 1$. We also evaluate the performance of RAFTEL (referred to as RAFTEL-Worst) in the worst case, *i.e.*, when the number of TEE replicas is zero with the system parameters set to $m = 0$ and $n = 3f + 1$. We evaluate RAFTEL, CHAINED-RAFTEL, and its counterparts in WAN with varying fault thresholds $f \in \{1, 2, 4, 8, 16, 32\}$. All protocols adopt 400 transactions per block and 256 B payload for each transaction. Due to space constraints, we defer the results of RAFTEL under varying parameters in both LAN and WAN (low-latency WAN setting), as well as the throughput vs. latency evaluation, to Appendix E.

Fig. 3a and Fig. 3b show the throughput and latency of the six protocols under varying fault thresholds f in a WAN setting. Since network communication is the dominant cost in WAN environments, the number of commit phases determines the latency and also the throughput due to serialized block generation. The throughput of Damysus and HotStuff is lower than RAFTEL, with HotStuff having the lowest throughput because it has the most commit phases. RAFTEL-Worst exhibits performance comparable to HotStuff, since both protocols have the same number of commit phases, while RAFTEL-Worst additionally introduces extra protocol-side decision logic. RAFTEL and CHAINED-RAFTEL achieve up to $1.02\times$ and $1.05\times$ higher throughput than HotStuff, with latency reduced by 61.46% and 62.84% respectively at $f = 32$. However, CHAINED-RAFTEL lags behind Achilles,

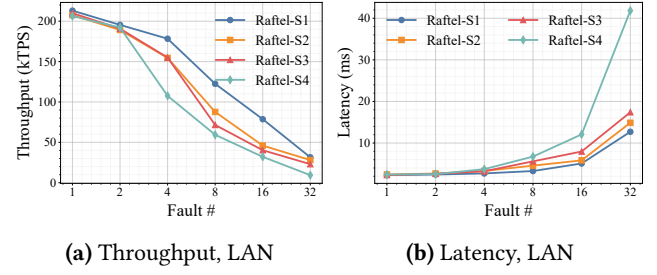


Figure 4. RAFTEL's performance under different scenarios.

showing 18.9% lower throughput and 21.2% higher latency under the same conditions. This gap stems from the system size: RAFTEL requires $3f + 1$ replicas, compared to $2f + 1$ in Achilles. Although only $f + 1$ votes are needed in RAFTEL, its leader must still broadcast proposals to $3f + 1$ replicas, incurring higher communication overhead.

7.3 Microbenchmark Evaluation

To evaluate the impact of performance acceleration caused by TEE replicas, we measure throughput and latency of RAFTEL under four scenarios:

- **RAFTEL-S1:** Leaders rotate among at least $f + 1$ TEE replicas (with $m = f + 1$) with both TEE-leader fast path and TEE-Quorum enabled (§4.3), yielding the best performance (as shown in prior experiments).
- **RAFTEL-S2:** Leaders rotate among TEE replicas, but fewer than $f + 1$ are available (*i.e.*, $m = f$). In this case, only the TEE-leader fast path is enabled.
- **RAFTEL-S3:** Leaders rotate among non-TEE replicas, while at least $f + 1$ TEE replicas exist in the system (*i.e.*, $m = f + 1$). Here, only the TEE-Quorum is enabled.
- **RAFTEL-S4:** Leaders rotate among non-TEE replicas, with fewer than $f + 1$ TEE replicas (*i.e.*, $m = f$). No optimization is enabled, and performance matches classical BFT protocols.

Fig. 4 illustrates the throughput and latency of RAFTEL under these four scenarios in LAN. The payload is 256 B, and the batch size is 400. RAFTEL-S1 achieves the highest throughput of 31.5 kTPS with 32 faults. Performance degrades in RAFTEL-S2 and RAFTEL-S3, with RAFTEL-S2 slightly outperforming RAFTEL-S3, indicating that the cost of a larger quorum size is smaller than the cost of an additional consensus phase. RAFTEL-S4 is the worst case, yielding the lowest throughput. Overall, the performance gap across the four scenarios is not large, and the difference becomes even less noticeable when the system size is small.

7.4 Performance Under Faults

We evaluate RAFTEL under crash faults in a WAN deployment with 25 replicas and a fault threshold of 8. We do not evaluate Byzantine equivocation faults, since RAFTEL handles equivocation in the same manner as conventional BFT

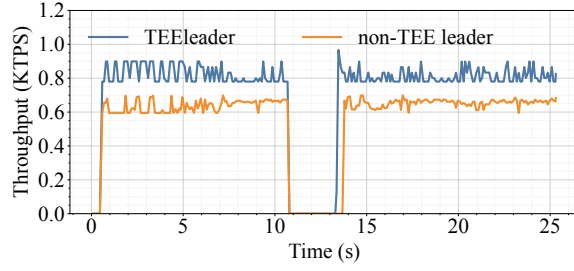


Figure 5. Performance of RAFTEL under faults.

Table 1. Overhead profiling for RAFTEL in LAN.

Protocols	Throughput (kTPS)			Latency (ms)		
	$f = 2$	$f = 4$	$f = 8$	$f = 2$	$f = 4$	$f = 8$
RAFTEL	192.4	151.6	89.7	2.1	2.6	4.5
RAFTEL-C	205.4	158.5	92.9	2.0	2.5	4.3

protocols and introduces no additional protocol-level differences in this case. We study the impact of crash faults on real-time throughput under two scenarios: crashes of the TEE leader and of a non-TEE leader. The view-change timeout is set to 2 seconds. Fig. 5 shows the average committed throughput over time. In both cases, the leader crashes at 10 seconds, causing the throughput to drop to nearly zero at 10.8 s. The view-change procedure is then triggered to recover from the failure. Recovery completes at 13.3 seconds for the TEE-leader crash and at 13.7 seconds for the non-TEE leader crash, after which throughput returns to the steady state. The faster recovery in the TEE-leader case is enabled by RAFTEL’s TEE-leader fast view-change.

7.5 Overhead Profiling

To understand the overhead of using SGX, we implement a variant of RAFTEL, called RAFTEL-C, which operates the trusted components outside the SGX enclave. On the critical path, a round of RAFTEL incurs 2 enclave interactions when the leader is TEE-enabled, and 4 enclave interactions when the leader is non-TEE, where each enclave interaction consists of one `ecall` and one `ocall`.

Table 1 shows the performance of RAFTEL and RAFTEL-C in a LAN setting with varying fault threshold $f \in \{2, 4, 8\}$. Each value is averaged over 5 runs; we report one decimal digit for readability. RAFTEL-C consistently outperforms RAFTEL, e.g., 205.4 kTPS vs. 192.4 kTPS and 2.0 ms vs. 2.1 ms latency given $f = 2$. The gap remains modest across all settings (3.4% to 6.7% throughput reduction and up to 0.2 ms latency increase), indicating that SGX introduces slight overhead.

7.6 KV-Store Application

We deploy Redis [58] on top of the protocols and use a workload consisting of 100% SET operations with 1 KB values. The

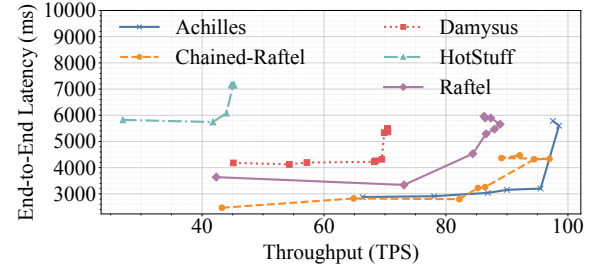


Figure 6. End-to-end Throughput vs. Latency.

fault threshold f is set to 8, and the batch size is 400. Fig. 6 illustrates the end-to-end latency (i.e., from when clients issue requests to when replies are received) and the corresponding throughput of the five protocols as the offered load increases until system saturation. The results show that the maximum throughput of RAFTEL and CHAINED-RAFTEL is 84 TPS and 92 TPS, respectively. RAFTEL significantly outperforms Damysus and HotStuff due to its quorum size of $f + 1$ and two-phase commit design. HotStuff achieves the lowest throughput (44 TPS) because it requires $3f + 1$ replicas and a quorum size of $2f + 1$. Achilles achieves the highest throughput (95 TPS) due to its minimized commit phase and smaller committee size. Overall, these results remain consistent with the micro-benchmark results, showing that the protocol-level optimizations of RAFTEL effectively translate into end-to-end application performance gains.

8 Conclusion

This paper revisits BFT consensus under heterogeneous trust. Instead of assuming that either all replicas are TEE-enabled or none are, we consider a partial-TEE setting where only a subset of replicas are equipped with TEEs. We formalize this setting through a universal partial-TEE model and show that heterogeneous trust fundamentally changes quorum formation and fault tolerance.

Our analysis establishes a tight resilience bound and reveals a threshold phenomenon: partial TEE deployment improves protocol efficiency immediately, but increases fault tolerance only after TEE replicas exceed a critical fraction of the system. Based on this insight, we design a dual-quorum construction and TEE-aware fast paths that safely exploit hardware-enforced non-equivocation. Building on these principles, we design and implement RAFTEL and CHAINED-RAFTEL, the first HotStuff-style protocols for universal partial-TEE deployments. Evaluation on Intel SGX shows that they significantly outperform classical BFT protocols and approach the performance of fully TEE-assisted designs in both LAN and WAN settings.

References

- [1] Mohammad Javad Amiri, Sujaya Maiyya, Divyakant Agrawal, and Amr El Abbadi. 2020. Seemore: A fault-tolerant protocol for hybrid cloud environments. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1345–1356. <https://doi.org/10.1109/ICDE48307.2020.00120>
- [2] Balaji Arun and Binoy Ravindran. 2022. Scalable Byzantine Fault Tolerance via Partial Decentralization. *Proc. VLDB Endow.* 15, 9 (2022), 1739–1752. <https://doi.org/10.14778/3538598.3538599>
- [3] Johannes Behl, Tobias Distler, and Rüdiger Kapitza. 2017. Hybrids on Steroids: SGX-Based High Performance BFT. In *Proc. of EuroSys*. <https://doi.org/10.1145/3064176.3064213>
- [4] Marcus Brandenburger, Christian Cachin, Rüdiger Kapitza, and Alessandro Sorniotti. 2019. Trusted Computing Meets Blockchain: Rollback Attacks and a Solution for Hyperledger Fabric. In *Proc. of SRDS*.
- [5] Ethan Buchman. 2016. *Tendermint: Byzantine Fault Tolerance in the Age of Blockchains*. Master's thesis. University of Guelph. <http://hdl.handle.net/10214/9769>
- [6] John Buford, Heather Yu, and Eng Keong Lua. 2009. *P2P networking and applications*. Morgan Kaufmann.
- [7] Miguel Castro and Barbara Liskov. 1999. Practical Byzantine Fault Tolerance. In *Proc. of OSDI*. <https://doi.org/10.1145/296806.296824>
- [8] Guoxing Chen, Sanchuan Chen, Yuan Xiao, Yinqian Zhang, Zhiqiang Lin, and Ten H Lai. 2019. SgxPectre: Stealing Intel secrets from SGX enclaves via speculative execution. In *Proc. of EuroSP*. <https://doi.org/10.1109/EuroSP.2019.00020>
- [9] Guoxing Chen and Yinqian Zhang. 2022. MAGE: Mutual Attestation for a Group of Enclaves without Trusted Third Parties. In *Proc. of USENIX Security*.
- [10] Arka Rai Choudhuri, Matthew Green, Abhishek Jain, Gabriel Kaptchuk, and Ian Miers. 2017. Fairness in an Unfair World: Fair Multiparty Computation from Public Bulletin Boards. In *Proc. of CCS*. <https://doi.org/10.1145/3133956.3134092>
- [11] Byung-Gon Chun, Petros Maniatis, Scott Shenker, and John Kubiatowicz. 2007. Attested Append-Only Memory: Making Adversaries Stick to Their Word. *SIGOPS Oper. Syst. Rev.* 41, 6 (2007), 189–204. <https://doi.org/10.1145/1294261.1294280>
- [12] Allen Clement, Flavio Junqueira, Aniket Kate, and Rodrigo Rodrigues. 2012. On the (limited) Power of Non-Equivocation. In *Proc. of PODC*. <https://doi.org/10.1145/2332432.2332490>
- [13] Allen Clement, Manos Kapritsos, Sangmin Lee, Yang Wang, Lorenzo Alvisi, Mike Dahlin, and Taylor Riche. 2009. Upright cluster services. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*. 277–290. <https://doi.org/10.1145/1629575.1629602>
- [14] Wikipedia contributors. [n. d.]. One Piece - Wikipedia. https://en.wikipedia.org/wiki/One_Piece. Retrieved May 2026.
- [15] Jérémie Decouchant, David Kozhaya, Vincent Rahli, and Jiangshan Yu. 2022. DAMYSUS: Streamlined BFT Consensus Leveraging Trusted Components. In *Proc. of EuroSys*. <https://doi.org/10.1145/3492321.3519568>
- [16] Jérémie Decouchant, David Kozhaya, Vincent Rahli, and Jiangshan Yu. 2024. OneShot: View-Adapting Streamlined BFT Protocols with Trusted Execution Environments. In *Proc. of IPDPS*. <https://doi.org/10.1109/IPDPS57955.2024.00095>
- [17] Determinant. [n. d.]. Salticidae: minimal C++ asynchronous network library. <https://github.com/Determinant/salticidae>. Retrieved May 2026.
- [18] Baltasar Dinis, Peter Druschel, and Rodrigo Rodrigues. 2023. RR: A fault model for efficient TEE replication. In *The Network and Distributed System Security Symposium*. Internet Society. <https://doi.org/10.14722/ndss.2023.24001>
- [19] Shuai Duan and Haibin Zhang. 2022. Foundations of Dynamic Byzantine Fault Tolerance. In *IEEE Symposium on Security and Privacy (S&P)*. 1317–1334. <https://doi.org/10.1109/SP46214.2022.9833787>
- [20] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the presence of partial synchrony. *J. ACM* 35, 2 (1988), 288–323. <https://doi.org/10.1145/42282.42283>
- [21] Fangyu Gai, Ali Farahbakhsh, Jianyu Niu, Chen Feng, Ivan Beschastnikh, and Hao Duan. 2021. Dissecting the Performance of Chained-BFT. In *Proc. of ICDCS*. <https://doi.org/10.1109/ICDCS51616.2021.00063>
- [22] Mingyuan Gao, Hung Dang, Ee-Chien Chang, and Jialin Li. 2022. Mixed Fault Tolerance Protocols with Trusted Execution Environment. *arXiv preprint* (2022). arXiv:2208.01946
- [23] Dimitra Giantsidi, Emmanouil Giortamis, Julian Pritzi, Maurice Bailleu, Manos Kapritsos, and Pramod Bhatotia. 2025. Recipe: Hardware-Accelerated Replication Protocols. *arXiv preprint* (2025). arXiv:2502.09251
- [24] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. 2017. Algorand: Scaling Byzantine Agreements for Cryptocurrencies. In *Proc. of SOSP*. <https://doi.org/10.1145/3132747.3132757>
- [25] Yangrui Guo, Qiangdong Yang, Hui Zhou, WeiQiang Lu, and Sheng Zeng. 2020. System and Methods for Selection and Utilizing a Committee of Validator Nodes in a Distributed System. Cypherium Blockchain. <https://github.com/cypherium/patent> Patent..
- [26] Suyash Gupta, Jelle Hellings, and Mohammad Sadoghi. 2021. RCC: Resilient concurrent consensus for high-throughput secure transaction processing. In *Proc. of ICDE*. <https://doi.org/10.1109/ICDE51399.2021.00124>
- [27] Suyash Gupta, Sajjad Rahnama, Shubham Pandey, Natacha Crooks, and Mohammad Sadoghi. 2023. Dissecting BFT Consensus: In Trusted Components We Trust!. In *Proc. of EuroSys*. <https://doi.org/10.1145/3552326.3587455>
- [28] Timo Hanke, Mahnush Movahedi, and Dominic Williams. 2018. DFINTY Technology Overview Series, Consensus System. *arXiv preprint* (2018). arXiv:1805.04548
- [29] Stephen Hemminger et al. 2005. Network Emulation with NetEm. In *Proc. of Linux.conf.au*.
- [30] Alexander Hentschel, Yahya Hassanzadeh-Nazarabadi, Ramtin Seraj, Dieter Shirley, and Layne Lafrance. 2002. Flow: Separating consensus and compute—block formation and execution. *arXiv preprint arxiv:2002.07403* (2002).
- [31] Matthew Hoekstra, Reshma Lal, Pradeep Pappachan, Vinay Phegade, and Juan Del Cuvillo. 2013. Using Innovative Instructions to Create Trustworthy Software Solutions. In *Proc. of HASP*. <https://doi.org/10.1145/2487726.2488370>
- [32] Heidi Howard, Fritz Alder, Edward Ashton, Amaury Chamayou, Sylvan Clebsch, Manuel Costa, Antoine Delignat-Lavaud, Cédric Fournet, Andrew Jeffery, Matthew Kerner, Fotios Kounelis, Markus A. Kuppe, Julien Maffre, Mark Russinovich, and Christoph M. Wintersteiger. 2023. Confidential Consortium Framework: Secure Multiparty Applications with Confidentiality, Integrity, and High Availability. *Proc. VLDB Endow.* 17, 2 (2023), 225–240. <https://doi.org/10.14778/3626292.3626304>
- [33] Intel Corporation. [n. d.]. Intel SGX OpenSSL. <https://github.com/intel/intel-sgx-ssl>. Retrieved May 2026.
- [34] Intel Corporation. 2025. 12th Generation Intel(R) Core(TM) Processor Datasheet. <https://www.intel.com/content/www/us/en/content-details/655258/12th-generation-intel-core-processors-datasheet-volume-1-of-2.html>. Retrieved May 2026.
- [35] Mohammad M Jalalzai, Jianyu Niu, Chen Feng, and Fangyu Gai. 2023. Fast-HotStuff: A Fast and Robust BFT Protocol for Blockchains. *IEEE Trans. Dependable Secure Comput.* 21, 4 (2023), 2478–2493. <https://doi.org/10.1109/TDSC.2023.3308848>
- [36] Simon Johnson, Vinnie Scarlata, Carlos Rozas, Ernie Brickell, and Frank McKeen. 2016. Intel Software Guard Extensions: EPID Provisioning and Attestation Services. *Intel Tech. Rep.* (2016), 119.
- [37] Rüdiger Kapitza, Johannes Behl, Christian Cachin, Tobias Distler, Simon Kuhnle, Seyed Vahid Mohammadi, Wolfgang Schröder-Preikschat,

- and Klaus Stengel. 2012. CheapBFT: Resource-Efficient Byzantine Fault Tolerance. In *Proc. of EuroSys*. <https://doi.org/10.1145/2168836.2168866>
- [38] Idit Keidar, Eleftherios Kokoris-Kogias, Oded Naor, and Alexander Spiegelman. 2021. All You Need is DAG. In *Proc. of PODC*. <https://doi.org/10.1145/3465084.3467905>
- [39] Rachel King, Quinn Burke, Yohan Beugin, Blaine Hoak, Kunyang Li, Eric Pauley, Ryan Sheatsley, and Patrick McDaniel. 2023. ParTEETor: A System for Partial Deployments of TEEs within Tor. In *Proc. of WPES*.
- [40] Ramakrishna Kotla, Lorenzo Alvisi, Mike Dahlin, Allen Clement, and Edmund Wong. 2010. Zyzzyva: Speculative Byzantine Fault Tolerance. *ACM Trans. Comput. Syst.* 27, 4, Article 7 (jan 2010), 39 pages. <https://doi.org/10.1145/1658357.1658358>
- [41] Jae Kwon and Ethan Buchman. 2016. Cosmos: A network of distributed ledgers.
- [42] XuperChain Lab. 2021. XuperChain Platform. <https://github.com/xuperchain/xuperchain>
- [43] Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine Generals Problem. *ACM Trans. Program. Lang. Syst.* 4, 3 (1982), 382–401. <https://doi.org/10.1145/357172.357176>
- [44] Dave Levin, John (JD) Douceur, Jay Lorch, and Thomas Moscibroda. 2009. TrInc: Small Trusted Hardware for Large Distributed Systems. In *Proc. of NSDI*. <https://doi.org/10.5555/1558977.1558978>
- [45] Jian Liu, Wenting Li, Ghassan O Karame, and N Asokan. 2018. Scalable Byzantine Consensus via Hardware-Assisted Secret Sharing. *IEEE Trans. Comput.* 68, 1 (2018), 139–151. <https://doi.org/10.1109/TC.2018.2860009>
- [46] Shengyun Liu, Paolo Viotti, Christian Cachin, Vivien Quéma, and Marko Vukolić. 2016. {XFT}: Practical fault tolerance beyond crashes. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. 485–500.
- [47] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Ivan Beschastnikh, Yinqian Zhang, Mohammad Sadeghi, and Chen Feng. 2025. Orthrus: Accelerating Multi-BFT Consensus Through Concurrent Partial Ordering of Transactions. In *Proc. of ICDE*. <https://doi.org/10.1109/ICDE6448.2025.00197>
- [48] Hanzheng Lyu, Shaokang Xie, Jianyu Niu, Chen Feng, Yinqian Zhang, and Ivan Beschastnikh. 2025. Ladon: High-Performance Multi-BFT Consensus via Dynamic Global Ordering. In *Proc. of EuroSys*.
- [49] J-P Martin and Lorenzo Alvisi. 2006. Fast Byzantine Consensus. *IEEE Trans. Dependable Secure Comput.* 3, 3 (2006), 202–215. <https://doi.org/10.1109/TDSC.2006.35>
- [50] Sinisa Matetic, Mansoor Ahmed, Kari Kostinen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels, and Srdjan Capkun. 2017. ROTE: Rollback Protection for Trusted Execution. In *Proc. of USENIX Security*. <https://doi.org/10.5555/3241189.3241289>
- [51] Fred J. Meyer and Dhiraj K. Pradhan. 2002. Consensus with dual failure modes. *IEEE Transactions on Parallel and Distributed Systems* 2, 2 (2002), 214–222. <https://doi.org/10.1109/71.89066>
- [52] Jianyu Niu, Wei Peng, Xiaokuan Zhang, and Yinqian Zhang. 2022. NARATOR: Secure and Practical State Continuity for Trusted Execution in the Cloud. In *Proc. of CCS*. <https://doi.org/10.1145/3548606.3560620>
- [53] Jianyu Niu, Xiaoqing Wen, Guanlong Wu, Shengqi Liu, Jiangshan Yu, and Yinqian Zhang. 2025. Achilles: Efficient TEE-Assisted BFT Consensus via Rollback Resilient Recovery. In *Proc. of EuroSys*. <https://doi.org/10.1145/3689031.3717457>
- [54] Bryan Parno, Jacob R. Lorch, John R. Douceur, James Mickens, and Jonathan M. McCune. 2011. Memoir: Practical State Continuity for Protected Modules. In *Proc. of S&P*. <https://doi.org/10.1109/SP.2011.38>
- [55] Daniel Porto, João Leitão, Cheng Li, Allen Clement, Aniket Kate, Flavio Junqueira, and Rodrigo Rodrigues. 2015. Visigoth fault tolerance. In *Proceedings of the Tenth European Conference on Computer Systems*. 1–14. <https://doi.org/10.1145/2741948.2741979>
- [56] Joshua Reynolds, Trevor Smith, Ken Reese, Luke Dickinson, Scott Ruoti, and Kent Seamons. 2018. A Tale of Two Studies: The Best and Worst of YubiKey Usability. In *Proc. of S&P*. <https://doi.org/10.1109/SP.2018.00067>
- [57] Mark Russinovich, Edward Ashton, Christine Avanesians, Miguel Castro, Amaury Chamayou, Sylvan Clebsch, Manuel Costa, Cédric Fournet, Matthew Kerner, Sid Krishna, et al. 2019. CCF: A Framework for Building Confidential Verifiable Replicated Services. Technical Report. Microsoft Research and Microsoft Azure.
- [58] Salvatore Sanfilippo and Pieter Noordhuis. 2025. Redis. <https://redis.io>. Accessed: 2026-05-11.
- [59] Michael Schwarz, Moritz Lipp, Daniel Moghimi, Jo Van Bulck, Julian Stecklina, Thomas Prescher, and Daniel Gruss. 2019. ZombieLoad: Cross-Privilege-Boundary Data Sampling. In *Proc. of CCS*. <https://doi.org/10.1145/3319535.3354252>
- [60] Marco Serafini, Péter Bokor, Dan Dobre, Matthias Majuntke, and Neeraj Suri. 2010. Scrooge: Reducing the costs of fast Byzantine replication in presence of unresponsive replicas. In *2010 IEEE/IFIP International Conference on Dependable Systems & Networks (DSN)*. IEEE, 353–362. <https://doi.org/10.1109/DSN.2010.5544295>
- [61] Amazon Web Services. [n.d.]. AWS Nitro Enclaves. <https://aws.amazon.com/ec2/nitro/>. Retrieved Jun, 2022..
- [62] Shweta Shinde, Zheng Leong Chua, Viswesh Narayanan, and Prateek Saxena. 2016. Preventing Page Faults from Telling Your Secrets. In *Proc. of AsiaCCS*. <https://doi.org/10.1145/2897845.2897885>
- [63] Rohit Sinha, Sivanarayana Gaddam, and Ranjit Kumaresan. 2019. Luciditee: A TEE-Blockchain System for Policy-Compliant Multi-party Computation with Fairness. *Cryptology ePrint Archive* (2019). iacr:2019/226
- [64] Alexander Spiegelman, Neil Girdharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. 2022. Bullshark: DAG BFT Protocols Made Practical. In *Proc. of CCS*. <https://doi.org/10.1145/3548606.3559361>
- [65] Chrysoula Stathakopoulou, Matej Pavlovic, and Marko Vukolić. 2022. State Machine Replication Scalability Made Simple. In *Proc. of EuroSys*. <https://doi.org/10.1145/3492321.3519579>
- [66] Raoul Strackx, Bart Jacobs, and Frank Piessens. 2014. ICE: A Passive, High-Speed, State-Continuity Scheme. In *Proc. of ACSAC*. <https://doi.org/10.1145/2664243.2664259>
- [67] Raoul Strackx and Frank Piessens. 2016. Ariadne: A Minimal Approach to State Continuity. In *Proc. of USENIX Security*. <https://doi.org/10.5555/3241094.3241162>
- [68] Philip Thambidurai and You-Keun Park. 1988. Interactive consistency with multiple failure modes. In *Proceedings [1988] Seventh Symposium on Reliable Distributed Systems*. IEEE, 93–100. <https://doi.org/10.1109/RELDIS.1988.25784>
- [69] The Diem Team. 2021. DiemBFT v4: State Machine Replication in the Diem Blockchain. *Technical Report* (2021). <https://developers.diem.com/papers/diem-consensus-state-machine-replication-in-the-diem-blockchain/2021-08-17.pdf>
- [70] Jo Van Bulck, Nico Weichbrodt, Rüdiger Kapitza, Frank Piessens, and Raoul Strackx. 2017. Telling Your Secrets Without Page Faults: Stealthy Page Table-Based Attacks on Enclaved Execution. In *Proc. of USENIX Security*. <https://doi.org/10.5555/3241189.3241271>
- [71] Stephan Van Schaik, Alyssa Milburn, Sebastian Österlund, Pietro Frigo, Giorgi Maisuradze, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. 2019. RIDL: Rogue In-Flight Data Load. In *Proc. of S&P*. <https://doi.org/10.1109/SP.2019.00087>
- [72] Wenhao Wang, Guoxing Chen, Xiaorui Pan, Yinqian Zhang, XiaoFeng Wang, Vincent Bindschaedler, Haixu Tang, and Carl A Gunter. 2017. Leaky Cauldron on the Dark Land: Understanding Memory Side-Channel Hazards in SGX. In *Proc. of CCS*. <https://doi.org/10.1145/3133956.3134038>
- [73] Weili Wang, Sen Deng, Jianyu Niu, Michael K. Reiter, and Yinqian Zhang. 2022. ENGRAFT: Enclave-Guarded Raft on Byzantine Faulty Nodes. In *Proc. of CCS*. <https://doi.org/10.1145/3548606.3560639>

- [74] Jan Werner, Joshua Mason, Manos Antonakakis, Michalis Polychronakis, and Fabian Monrose. 2019. The Severest of Them All: Inference Attacks Against Secure Virtual Enclaves. In *Proc. of AsiaCCS*. <https://doi.org/10.1145/3321705.3329820>
- [75] Yuanzhong Xu, Weidong Cui, and Marcus Peinado. 2015. Controlled-Channel Attacks: Deterministic Side Channels for Untrusted Operating Systems. In *Proc. of S&P*. <https://doi.org/10.1109/SP.2015.45>
- [76] Sravya Yandamuri, Ittai Abraham, Kartik Nayak, and Michael K. Reiter. 2023. Communication-Efficient BFT Using Small Trusted Hardware to Tolerate Minority Corruption. In *Proc. of OPODIS*. <https://doi.org/10.4230/LIPIcs.OPODIS.2022.24>
- [77] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT Consensus with Linearity and Responsiveness. In *Proc. of PODC*. <https://doi.org/10.1145/3293611.3331591>

A BFT consensus Background

Classical BFT consensus. BFT consensus protocols (e.g., PBFT [7] and Zyzzyva [40]) have been cornerstone primitives for building Byzantine state machine replication (SMR) for decades. They can tolerate up to one-third Byzantine faults and typically require at least two rounds of communication among nodes to reach agreement. To reduce the communication phase, protocols like FaB [49] explore reducing the number of rounds to one, but this comes at the cost of stricter resilience requirements (e.g., needing $5f + 1$ nodes to tolerate f faults). The rise of blockchains has fueled a new wave of BFT protocols with a focus on scalability and security. Chain-based BFT designs, such as Tendermint [5] and HotStuff [77], streamline leader rotation and pipeline block commitments, becoming the backbone of many blockchain platforms [25, 30, 41, 42]. Due to its promising features, we chose to customize HotStuff [77] when TEE nodes are not available. We also note that some state-of-the-art protocols, such as Multi-BFT consensus [2, 26, 47, 48, 65] and DAG-based BFT consensus [38, 64], have better performance. Our results may also be extended to them. More recently, Multi-BFT consensus [2, 26, 47, 48, 65] and DAG-based BFT consensus [38, 64] further replace linear chains with multiple parallel chain structures or directed acyclic graphs, allowing replicas to confirm transactions along multiple paths, reducing latency, or even better tolerating network asynchrony.

BFT consensus Using Small Trusted Hardware. Unlike TEEs that support computing arbitrary functions, small trusted hardware [76] provides small trusted abstractions such as an append-only log and a monotonic counter. Small trusted hardware with a small Trusted Computing Base (TCB) can be realized by Trusted Platform Modules (TPMs) [54, 66, 67] and YubiKey [56]. Chun et al. [11] pioneered the usage of trusted logs to prohibit Byzantine behaviors (e.g., proposal and vote equivocation), which improves the fault tolerance of corrupted replicas from one-third to the minority. Levin et al. [44] simplify the trusted log abstraction to a trusted persistent counter within the same security guarantee. Later, MinBFT [44] and CheapBFT [37] further advance system performance by optimizing the fast path and happy path. Recently, Yandamuri et al. [76] improve the resilience of HotStuff from one-third to $1/2 - \epsilon$, while keeping a total of $O(n)$ communication per view in a partially synchronous network.

B Upper Bound of Faults

In this section, we establish an upper bound on the number of Byzantine faults tolerable in the universal partial-TEE model.

Let $n = m + k$, where m is the number of TEE replicas and k is the number of non-TEE replicas. Up to f replicas may be Byzantine. We consider *single-shot Byzantine agreement* (BA)

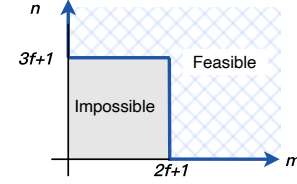


Figure 7. Feasible and infeasible regions for Byzantine agreement, where m denotes the number of TEE replicas and n denotes the total number of replicas in the system.

under partial synchrony. A Byzantine agreement protocol must satisfy the following properties:

- **Agreement.** No two honest replicas decide differently.
- **Validity.** If all honest replicas start with the same input v , then any honest replica that decides must decide v .
- **Termination.** Every honest replica eventually decides.

Fig. 7 shows the impossibility result for BA.

Theorem B.1. *There exists no protocol Π that solves Byzantine agreement in a partially synchronous network under the universal partial-TEE model if*

$$f \geq \max\left\{\frac{n}{3}, \frac{m}{2}\right\}.$$

We prove the theorem via a reduction to a minimal three-replica impossibility.

B.1 Base Impossibility

Lemma B.2. *There exists no protocol that solves Byzantine agreement in a partially synchronous network with $n = 3$, $f = 1$, and at least one non-TEE replica.*

Proof. Assume for contradiction that such a protocol Π exists.

Consider three replicas 1, 2, 3, where replica 3 is a non-TEE replica and may be Byzantine. We construct three executions.

Execution E_A . Replicas 1 and 3 have input A , and replica 2 is Byzantine and sends no messages. By **Validity** and **Termination**, replica 1 must eventually decide A .

Execution E_B . Replicas 2 and 3 have input B , and replica 1 is Byzantine and sends no messages. By **Validity** and **Termination**, replica 2 must eventually decide B .

Execution E . Replica 1 has input A , replica 2 has input B , and replica 3 is Byzantine. Before GST, the adversary delays all messages between replicas 1 and 2. Replica 3 behaves as follows: toward replica 1, it simulates an honest replica with input A ; toward replica 2, it simulates an honest replica with input B .

Then, the local view of replica 1 in E is indistinguishable from its view in E_A , so replica 1 must decide A . Similarly, the local view of replica 2 in E is indistinguishable from its view in E_B , so replica 2 must decide B .

Let GST be any time after both decisions occur. This yields a valid partially synchronous execution in which two honest

replicas decide differently, violating **Agreement**. This is a contradiction. $\square$

B.2 Reduction via Partitioning

Lemma B.3. *Suppose the n replicas can be partitioned into three non-empty disjoint sets S_1, S_2, S_3 such that:*

- $|S_i| \leq f$ for all $i \in \{1, 2, 3\}$, and
 - one set (say S_3) contains only non-TEE replicas.
- Then no protocol can solve Byzantine agreement in this system.*

Proof. Assume for contradiction that such a protocol exists.

We construct a reduced three-party execution in which each set S_i behaves as a single *virtual replica*. Before GST, the adversary delays all messages between different sets arbitrarily. In particular, all replicas within the same set receive identical external messages from other sets. Moreover, replicas within the same set start from the same local state and input. Since protocol Π is deterministic, replicas within the same set evolve identically and send identical messages to replicas outside the set. Therefore, from the perspective of the other sets, each set S_i is indistinguishable from a single replica executing protocol Π .

In particular, the adversary corrupts S_3 . Because S_3 contains only non-TEE replicas, it can equivocate arbitrarily and send inconsistent messages to S_1 and S_2 . Thus, the system behaves like a three-replica system with one Byzantine replica capable of equivocation, which exactly matches the setting of Lemma B.2. This is impossible, yielding a contradiction. $\square$

B.3 Proof of Theorem B.1

Proof. We consider two regimes depending on which term dominates

$$\max \left\{ \frac{n}{3}, \frac{m}{2} \right\}.$$

Case 1: $m/2 \leq n/3$ (**mixed-dominated regime**). In this regime,

$$\max \left\{ \frac{n}{3}, \frac{m}{2} \right\} = \frac{n}{3},$$

so the theorem assumption implies

$$f \geq n/3,$$

or equivalently,

$$n \leq 3f.$$

Moreover,

$$m \leq 2n/3,$$

which implies

$$k = n - m \geq n/3.$$

Hence, there are at least $n/3$ non-TEE replicas.

When $n = 3$, we have $m \leq 2$, so Lemma B.2 directly applies.

Now consider $n > 3$. Construct a set S_3 consisting of $\lceil n/3 \rceil$ non-TEE replicas, which is possible since $k \geq n/3$. Partition

the remaining replicas into two non-empty sets S_1 and S_2 such that

$$|S_1|, |S_2| \leq \lceil n/3 \rceil.$$

Since $f \geq n/3$, all three sets satisfy $|S_i| \leq f$. By construction, S_3 contains only non-TEE replicas. Therefore, the conditions of Lemma B.3 hold, and Byzantine agreement is impossible.

Case 2: $m/2 > n/3$ (**TEE-dominated regime**). In this regime,

$$\max \left\{ \frac{n}{3}, \frac{m}{2} \right\} = \frac{m}{2},$$

so the theorem assumption implies

$$f \geq m/2,$$

or equivalently,

$$m \leq 2f.$$

Moreover,

$$\frac{m}{2} > \frac{m+k}{3}$$

implies

$$m > 2k.$$

If $k = 0$, the system reduces to the pure-TEE setting, where Byzantine agreement is impossible under $f \geq n/2$ by known lower bounds [12].

Now consider $k > 0$. Since $m > 2k$, we have $m \geq 3$.

Let S_3 be the set of all non-TEE replicas, so that

$$|S_3| = k < m/2 \leq f.$$

Partition the TEE replicas into two non-empty sets S_1 and S_2 such that

$$|S_1|, |S_2| \leq f,$$

which is always possible since $m \leq 2f$.

Thus, all three sets are non-empty, each has size at most f , and S_3 contains only non-TEE replicas. Therefore, the conditions of Lemma B.3 hold, and Byzantine agreement is impossible. $\square$

C Proof of Correctness

C.1 RAFTEL

Recall that the system consists of n replicas with m TEE replicas and k non-TEE replicas, where $n = m + k$. There are at most f faulty replicas, bounded by Theorem B.1. The *TEE-Quorum* size is defined as $Q_T = \max \left\{ \left\lfloor \frac{m}{2} \right\rfloor + 1, f + 1 \right\}$, and the *Mixed-Quorum* size is defined as $Q_M = n - f$. We generalize the quorum-intersection guarantee in the following lemma.

Lemma C.1 (Quorum Intersection). *Under the settings of $m \geq 2f + 1$ or $n \geq 3f + 1$, any two valid quorums—whether two TEE-Quorums, two Mixed-Quorums, or one of each—must intersect in at least one replica that does not equivocate (i.e., never votes for two conflicting messages).*

Proof. Let S ($|S| = n$) be the set of all the replicas, S_{TEE} be the set of all the TEE replicas. We have $|S_{TEE}| = m$. We consider the following three cases:

Case 1: Mixed vs. Mixed. For any two sets that form a Mixed-Quorum $S_{M1}, S_{M2} \subseteq S$, we have

$$|S_{M1} \cap S_{M2}| \geq |S_{M1}| + |S_{M2}| - n = n - 2f.$$

When $n \geq 3f + 1$, the intersection size is at least $f + 1$. Since there are at most f faulty replicas, $S_{M1} \cap S_{M2}$ must contain at least one honest replica.

When $n < 3f + 1$, the classical honest-intersection argument no longer applies. However, in this regime we have $m \geq 2f + 1$. The number of non-TEE replicas is $k = n - m$. Since $m \geq 2f + 1$, we have

$$k = n - m \leq n - (2f + 1) = n - 2f - 1.$$

Therefore,

$$|S_{M1} \cap S_{M2}| \geq n - 2f > k,$$

which implies that $S_{M1} \cap S_{M2}$ must contain at least one TEE replica.

Case 2: TEE vs. Mixed. For any two sets that forms a TEE-Quorum and Mixed-Quorum respectively, i.e., $S_T \subseteq S_{TEE}, S_M \subseteq S$, we have

$$|S_T \cap S_M| \geq |S_T| + |S_M| - n \geq (f + 1) + (n - f) - n = 1.$$

Since all the replicas in S_T are TEE replicas, $S_T \cap S_M$ contains at least one TEE replica that cannot equivocate.

Case 3: TEE vs. TEE. Let $S_{T1}, S_{T2} \subseteq S_{TEE}$ be two TEE-Quorums. We have

$$|S_{T1} \cap S_{T2}| \geq |S_{T1}| + |S_{T2}| - m \geq 2 \lfloor \frac{m}{2} \rfloor + 2 - m \geq 1.$$

Similarly, since the replicas in S_{T1} and S_{T2} are all TEE replicas, $S_{T1} \cap S_{T2}$ contains at least one TEE replica that cannot equivocate.

Therefore, any two valid quorums must intersect in at least one non-equivocating replica: either an honest replica (which only signs one block per view) or a TEE replica (which is hardware-enforced to do so). $\square$

Lemma C.2 (Prepared Block Uniqueness). *If two blocks b and b' are prepared in the same view v , $b = b'$.*

Proof. Suppose that L is the leader of view v . We consider two cases:

Case 1: L is a TEE replica. For a block proposed by TEE Leader to be prepared, it must be through **TEEprepare**, which can be invoked at most once per view. **TEEprepare** binds the proposal to (v, phase) and a generate prepare certificate, enforcing non-equivocation. Hence, L can propose at most one block in view v . Backups mark a block prepared only upon receiving the leader's valid prepare certificate for v , so there is a unique prepared block in view v . Therefore $b = b'$.

Case 2: L is a non-TEE replica. Assume, for contradiction, that $b \neq b'$ and both become prepared in view v . Then there

exist two valid prepare certificates qc_b and $qc_{b'}$ for b and b' , respectively, formed in view v under the dual-quorum (each is either a TEE-Quorum or a Mixed-Quorum certificate). By Lemma C.1 (Quorum Intersection), the two quorums underlying qc_b and $qc_{b'}$ intersect in at least one replica n that cannot equivocate within view v . Consequently, n cannot have signed prepare votes for two different block hashes in the same view, contradicting the existence of distinct QC_b and $QC_{b'}$. Hence $b = b'$.

In all cases, at most one block can be prepared in a given view, which proves the claim. $\square$

Lemma C.3 (No Equivocation). *Let v be a view and $v' \leq v$ be the latest view in which a block b was stored as prepared. Then, any block/view pair (b', v') prepared in some view $v'' \geq v'$ must extend b , i.e. $b' \succ^* b$.*

Proof. We prove this by induction on the view number v .

Base Case. In view 0, only the genesis block is prepared, and any proposed block must extend the genesis block. Thus, the property holds trivially.

Inductive Case. Assume that the property holds up to view v (induction hypothesis, IH). We will show that it also holds for view $v + 1$. Let $v' \leq v$ be the latest view in which a block b was prepared, and let L be the leader of view $v + 1$. We distinguish two cases:

Case 1: TEE leader. If L is a TEE replica, then during the new-view phase, it received a *quorum* of new-view messages, each carrying the latest prepared block known to that replica. By quorum intersection (Lemma C.1), at least one non-equivocating replica in this set reported block b or an extension of b . The TEE leader must select the highest-view prepared block among the collected certificates (via the **TEEprepare**) and can invoke **TEEprepare** only once per view, which binds block b with view $v + 1$. Thus, L must therefore propose a unique block b' in view $v + 1$, and b' necessarily extends the selected highest prepared block, which by IH extends b . Hence $b' \succ^* b$.

Case 2: Non-TEE leader. If L is a non-TEE replica, then a block becomes prepared only if it gathers *quorum* votes from arbitrary replicas. In the new-view phase, the leader must also collect a *quorum* of new-view messages. By quorum intersection (i.e., Lemma C.1), there exists at least one replica that cannot equivocate in the intersection of this set which voted for the latest prepared block b . By the induction hypothesis, any honest replica only votes for an extension of b . Therefore, any newly prepared block b' in view $v + 1$ must satisfy $b' \succ^* b$.

In both cases, any prepared block in view $v + 1$ extends the latest prepared block from view $\leq v$. Hence, the invariant holds by induction. $\square$

Theorem C.4 (Safety). *Honest replicas never execute conflicting blocks.*

Proof. Suppose that an honest replica n_1 executes block b_1 in view v_1 ; thus, the block must be prepared in that view. According to Lemma C.2, block b_1 is the only block that is prepared in view v_1 . Next, we will prove, by a complete induction on the number of views between v_1 and v_2 , that for any view $v_2 \geq v_1$, if block b_2 is stored as prepared in view v_2 , then $b_2 >^* b_1$.

Base case $v_1 = v_2$. Since b_1 is prepared in v_1 and an honest replica only prepares (and thus executes) a unique block per view, any prepared block b_2 in view v_1 must satisfy $b_2 = b_1$, hence $b_2 >^* b_1$.

Inductive step. Assume that the statement holds for every view strictly smaller than v_2 (induction hypothesis, IH). Let b_2 be a block prepared in view $v_2 > v_1$. Let b be the (uniquely) latest block that was stored as prepared in some view v with $v_1 \leq v < v_2$. By the IH, we have $b >^* b_1$. By Lemma C.3 (Safe Prepared Blocks), any block prepared in a view $v_2 \geq v$ must extend b , hence $b_2 >^* b$. By IH, $b >^* b_1$. Transitivity of $>^*$ gives $b_2 >^* b_1$.

We have shown that any block b_2 prepared (hence executable) in any later view $v_2 \geq v_1$ extends b_1 . Therefore, if another honest replica i_2 executes a block b_2 in some view $v_2 \geq v_1$, then $b_2 >^* b_1$, so b_1 and b_2 cannot be conflicting. This proves that honest replicas never execute conflicting blocks. $\square$

Theorem C.5 (Liveness). *Clients' transactions will eventually be included in a block committed by honest replicas.*

Proof. Without loss of generality, we assume the leader of view v is honest after GST. There are two cases for the NEW-VIEW phase.

- If the leader of view v receives a prepare certificate or a block proposed by the TEE leader from the previous view for a block b , then the leader proposes a block b' that extends b . All honest replicas will accept and store the block since it is for the latest view. The leader can collect certificates from *quorum* replicas because at least the *quorum* honest replicas will send theirs and can form a *quorum* certificate, and then send this QC to all replicas. After one (TEE leader) or three (non-TEE leader) rounds of this process, all honest replicas will execute the block once they have pulled all previous blocks.
- If the leader receives *quorum* view certificates, it selects the prepared block b with the highest view. Then, it extends block b with its block b' . After that, at least all *quorum* honest replicas will store and vote for b' . When receiving *quorum* certificates, the leader will prepare a *quorum* certificate for b' and broadcast the certificate.

In both cases, the leader can coordinate with other replicas to commit a new block including honest clients' transactions. This completes the proof. $\square$

C.2 CHAINED-RAFTEL

CHAINED-RAFTEL differs from RAFTEL by pipelining the communication phases. Thus, the quorum-intersection result (Lemma C.1) and the prepared block uniqueness (Lemma C.2) also hold for CHAINED-RAFTEL because of the same Dual quorum rule and block prepare rule.

Theorem C.6 (Safety of CHAINED-RAFTEL). *In CHAINED-RAFTEL, honest replicas never execute conflicting blocks.*

Proof Sketch. Suppose an honest replica executes block b_1'' in view v_1 , and another executes block b_2'' in view $v_2 \geq v_1$. In CHAINED-RAFTEL, a block can only be executed once it forms a one-chain (with TEE leader) or three-chain (with Non-TEE leader). Then we consider two cases: If $v_1 = v_2$, according to Lemma C.3, $b_1'' = b_2''$. If $v_2 > v_1$, we consider two cases:

Case 1: TEE leader. TEEprepare function ensures that the TEE leader extends block b_1'' and proposes a unique block in this view. By induction across views, every subsequently executed block extends b_1'' , and thus two conflicting blocks can never both be executed.

Case 2: Non-TEE leader. Quorum intersection guarantees that the leader of v_2 receives information from at least one honest replica that prepared b_1'' (or its successor), forcing the new block to extend it. By induction across views, every subsequently executed block extends b_1'' , and thus two conflicting blocks can never both be executed. $\square$

Theorem C.7 (Liveness of CHAINED-RAFTEL). *Clients' transactions will eventually be included in a block committed by correct replicas.*

Proof Sketch. After GST, if the leader L of view v is honest, it can gather a *quorum* of valid new-view messages (either a TEE-Quorum or a Mixed-Quorum). If a prepare certificate (or a TEE-leader proposal) in the previous view is available, L extends that block; otherwise, from the collected new-view messages L selects the highest prepared block and extends it. The proposal is broadcast, and all honest replicas accept and vote for it since quorum intersection guarantees the proposal extends the honest chain. L then aggregates a *valid voting quorum* into a certificate and disseminates it.

If L is a TEE leader, the proposed block is committed through the fast path of one-chain commit rule; otherwise, the block is committed following the three-chain commit rule. In all cases, the block (containing pending transactions) is prepared and then committed. Therefore, honest clients' transactions will eventually be included in some committed block, establishing liveness. $\square$

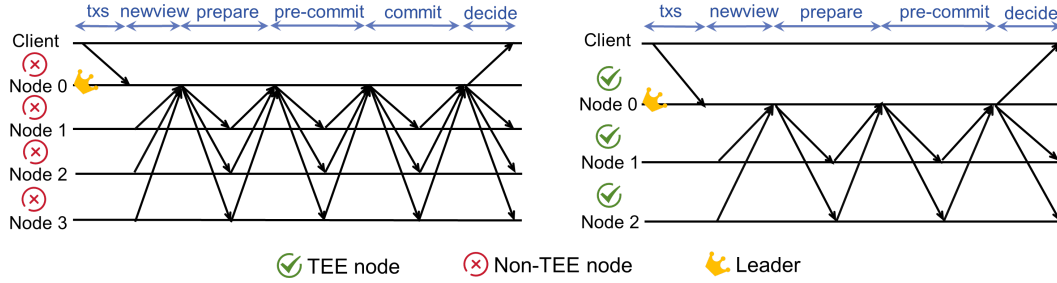


Figure 8. The overviews of HotStuff (left) and Damysus (right).

D HotStuff and Damysus in a Nutshell

D.1 HotStuff

HotStuff is a leader-based BFT protocol that adopts a chain structure and supports frequent leader rotation. It runs with $n = 3f + 1$ replicas and uses quorum certificates (QCs) that aggregate $n - f$ votes. A chained variant of HotStuff applies pipelining structures to improve throughput.

Protocol description. HotStuff has three communication phases, *i.e.*, PREPARE, PRE-COMMIT, and COMMIT, to commit transactions. Each phase has two communication steps: the leader broadcasts to all backups and then receives their votes to generate the QC, as shown in Fig. 8. Specifically, a *PrepareQC* certifies that a block has received a quorum of votes in the PREPARE phase, a *PrecommitQC* certifies the quorum votes in the PRE-COMMIT phase, and a *CommitQC* certifies the quorum votes in the COMMIT phase. In addition, HotStuff uses a NEW-VIEW phase for leader rotation and a DECIDE phase for block execution and client replies. The detailed procedure is described as follows.

- ❶ In the NEW-VIEW phase, each backup increments its view and sends a new-view message carrying its highest observed *prepareQC* to the leader.
- ❷ In the PREPARE phase, the leader waits for $n - f$ new-view messages and computes the *highQC* as the QC with the highest view, then it proposes a block that extends the replica justified by *highQC* and broadcasts the block proposal together with *highQC*. Upon receipt, each backup replica checks: it votes only if the proposal either extends its locked branch or carries a QC with a view higher than its current lock; otherwise, it withholds the vote. Backup replicas that pass the check return a PREPARE vote.
- ❸ In the PRE-COMMIT phase, the leader waits for $n - f$ PREPARE votes and assembles them into a *prepareQC* (threshold-signed). The leader then broadcasts this certificate. After seeing a valid *prepareQC* for the current view, backup replicas send a PRE-COMMIT vote.
- ❹ In the COMMIT phase, the leader collects $n - f$ PRE-COMMIT votes to form a *precommitQC* and broadcasts it. Upon receiving this certificate, replicas update *lockedQC* to *precommitQC*,

and then send a COMMIT vote. This lock prevents future votes that would conflict with the locked branch.

- ❺ In the DECIDE phase, after receiving $n - f$ COMMIT votes, the leader forms a *commitQC* and broadcasts a decide message so that backups commit and execute the block and reply to the client.

D.2 Damysus

Damysus is a TEE-assisted BFT protocol built atop HotStuff. It introduces two trusted components, the CHECKER and the ACCUMULATOR, which raise the fault tolerance from $n = 3f + 1$ to $n = 2f + 1$, and cut one communication phase, as shown in Fig. 8. Damysus also has a chained version that uses pipelining to achieve higher performance.

Trusted components. The two trusted components, CHECKER and ACCUMULATOR, are introduced as follows.

- **CHECKER.** The CHECKER maintains a monotonic counter to record the current view and phase. It also stores a pair consisting of the view number and the hash of the last prepared block. This pair is included in the commitment that backup replicas (*i.e.*, non-leader replicas) send to the leader during the NEW-VIEW phase. The monotonic counter prevents Byzantine replicas from equivocating messages (*e.g.*, blocks or votes).
- **ACCUMULATOR.** The ACCUMULATOR is used by the leader after collecting $f + 1$ commitments from backup replicas in the NEW-VIEW phase. It outputs the view number and hash of the prepared block with the highest view among these $f + 1$ blocks. The leader then extends this block to ensure safety.

In Damysus, the CHECKER uses a monotonic counter to link each proposal or vote with a unique (*view, phase*) identifier, which prevents equivocation. It further records the view number and hash of the latest prepared block; backup replicas attach this information to their new-view commitments when reporting to the leader. The ACCUMULATOR is used by the leader when processing new-view messages: after receiving $f + 1$ commitments, it selects the prepared block with the highest view and outputs its (*view, hash*) pair.

Protocol description. Unlike HotStuff, which requires three phases (PREPARE, PRE-COMMIT, and COMMIT), Damysus reduces the process to two phases: PREPARE and PRE-COMMIT. Each phase still consists of two steps—the leader broadcasts to all backups and collects their votes. Besides, Damysus retains the NEW-VIEW phase for leader rotation and the DECIDE phase for block execution and client replies. Including the step where a client sends its transactions, the protocol requires six communication steps to finalize a commitment (excluding the NEW-VIEW phase). In comparison, HotStuff introduces an extra COMMIT phase, resulting in a total latency of eight steps.

- ❶ In the NEW-VIEW phase, each backup increments its view and sends its commitment, which contains its (view, hash) pair stored in the CHECKER, to the leader.
- ❷ In the PREPARE phase, the leader generates the latest prepared block from $f + 1$ received new-view messages using the ACCUMULATOR. The leader then extends the latest prepared block certified by the CHECKER. Upon receiving this signed block from the leader, each backup responds with a vote generated by the CHECKER. In Damysus, a CHECKER's counter is incremented each time a CHECKER is called.
- ❸ In the PRE-COMMIT phase, the leader collects $f + 1$ prepare votes from backups, and broadcasts a combined version to backups. Backups consider the proposed block as prepared, store the view number and hash value associated with the block via the CHECKER, and reply with a vote it generates.
- ❹ In the DECIDE phase, the leader collects $f + 1$ commit votes from backups and broadcasts a combined version to backups so that they can execute the block. Every replica verifies the authenticity of the received messages signed by trusted components before processing them.

One-phase optimization. Achilles [53] demonstrates that the PREPARE phase in Damysus can be eliminated by exploiting equivocation prevention and chained commitment. Specifically, equivocation prevention is guaranteed by TEE-based trusted components, whereas chained commitment ensures that once descendant blocks are committed, their uncommitted parents are also finalized. With these mechanisms, Achilles employs customized chained commit rules to achieve reduced message complexity. In this paper, we adopt these optimizations and then propose a customized trusted component, called CHECKER+, to reduce the one-phase from Damysus.

E Additional Experiments

To complement the scalability results under the standard WAN (RTT 100 ± 5 ms) presented in §7.2, we provide additional micro-benchmarks in a *low-latency WAN* setting (RTT 40 ± 2 ms) and in LAN (RTT 0.1 ± 0.02 ms). The low-latency WAN setting isolates the impact of protocol-phase reduction when network delay is less dominant. All experiments

retain the same hardware setup described in §7.1. We vary batch sizes of 200, 400, and 600, transaction payloads of 0 B, 256 B, and 512 B, and fault thresholds $f \in \{1, 2, 4, 8, 16, 32\}$. Payloads of 0 B and batch size of 400 transactions are used to evaluate the protocols' overhead, while other sets of payloads and transaction numbers have been selected to observe the trend when increasing the size of blocks.

E.1 Performance in WAN

We evaluate RAFTEL in WAN with varying parameters.

1) *Varying fault thresholds.* Figs. 9a and 9b show throughput and latency in the low-latency WAN setting. Compared with the standard WAN (§7.2), absolute throughput is higher across all protocols because the smaller RTT reduces the cost of each communication phase. Notably, the *relative* speed-up of RAFTEL and CHAINED-RAFTER over HotStuff increases to 2.0 $\times$ and 2.8 $\times$ at $f = 32$, respectively, with latency reduced by 71.2% and 67.6%. The larger gap stems from the same system-size effect: RAFTEL's leader must broadcast proposals to all $3f + 1$ replicas regardless of quorum size. Under lower RTT, this communication penalty shrinks, making its fewer commit phases more impactful.

2) *Varying payload size.* Fig. 9e and 9f show the performance results of five protocols with varying payload sizes. The payloads are 0 B, 256 B, and 512 B. The number of faults is 8, and the batch size is fixed at 400. The experimental results indicate that as the payload increases from 0 B to 512 B, the throughput of CHAINED-RAFTER and Achilles decreases by approximately 50%, while RAFTEL and HotStuff decrease by about 30%. In contrast, Damysus shows only a negligible reduction of 2%. With respect to latency, CHAINED-RAFTER increases by 66%, RAFTEL increases by 122%, Damysus increases by merely 2%, Achilles increases by 96%, and HotStuff increases by 47%.

3) *Varying batch size.* Fig. 9k and 9l illustrate the impact of varying batch sizes on the performance of five protocols. The number of faults is 10, the payload is 256 B, and the batch size varies from 200, 400, to 600. As the batch size increases from 200 to 600, the throughput improves substantially: CHAINED-RAFTER increases by 100%, RAFTEL by 205%, Damysus by 193%, Achilles by 180%, and HotStuff by 125%. Latency also shows a slight upward trend, with the latency of CHAINED-RAFTER increasing by 60%, RAFTEL first decreasing by 16% and then increasing by 28%, Damysus rising by about 2%, Achilles first decreasing by 40% and then increasing by 20%, and HotStuff increasing by 33%. This shows that the increase in batch size significantly boosts the throughput of the five protocols while also causing a slight increase in latency.

E.2 Performance in LAN

To minimize the effect of network communication, we also evaluate RAFTEL in LAN.

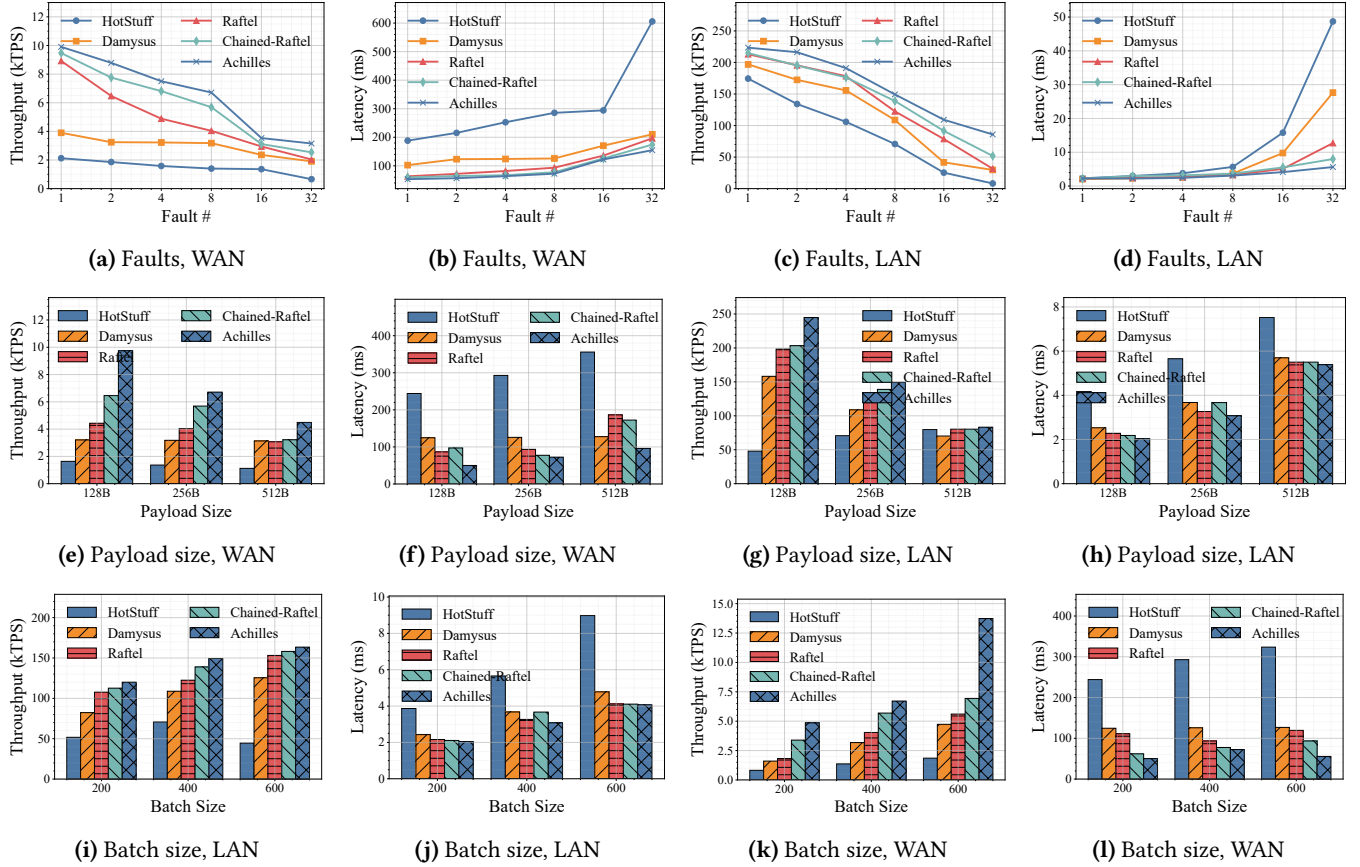


Figure 9. Throughput and latency of RAFTEL with varying parameters in WAN and LAN.

1) *Varying fault thresholds.* Fig. 9c and Fig. 9d report the throughput and latency results. Compared to the WAN setting, the performance gap between HotStuff/Damysus, and the more optimized protocols narrows, since the cost of additional commit phases is less pronounced. For $f = 32$, CHAINED-RAFTEL achieves throughput $1.1\times$ – $2.2\times$ higher than Damysus and $1.2\times$ – $6.3\times$ higher than HotStuff. As the fault threshold increases from 1 to 32, throughput decreases for all protocols but at different rates: RAFTEL and Damysus degrade by $5.8\times$ and $5.7\times$, respectively, while CHAINED-RAFTEL shows a more moderate $3.1\times$ decrease. By contrast, HotStuff suffers the steepest decline ($20.2\times$), whereas Achilles is the most resilient ($1.6\times$). Thus, RAFTEL and CHAINED-RAFTEL are more scalable than HotStuff and Damysus. Although Achilles still delivers the best performance—reaching 85.8 kTPS throughput and 5.6 ms latency at $f = 32$ —the performance gap between CHAINED-RAFTEL and Achilles is notably smaller in LAN than in WAN.

2) *Varying payload size.* Fig. 9g and 9h show the performance results of five protocols with varying payload sizes in the LAN setting. The experimental configuration is identical to that in WAN. Overall, the trends observed in LAN are consistent with those in WAN: CHAINED-RAFTEL and Achilles

exhibit the most significant degradation, while RAFTEL, HotStuff, and Damysus also show performance reductions, though to a lesser extent. Compared with WAN, however, the magnitude of variation in LAN is notably smaller, indicating that the impact of payload size is more limited in the LAN setting.

3) *Varying batch size.* Fig. 9i and 9j illustrate the performance of five protocols with varying batch sizes in LAN. The settings are the same as those in WAN. Similar to the WAN results, increasing the batch size improves throughput while causing only slight changes in latency. Nevertheless, the improvements in LAN are relatively less pronounced, and the latency fluctuations are more modest, reflecting the reduced sensitivity to batch size in low-latency environments.

E.2.1 Throughput vs. latency. Fig. 10 illustrates the end-to-end latency (*i.e.*, from when clients create transactions to when replies are received) and the corresponding throughput of the five protocols as the offered load increases until system saturation. The fault threshold f is set to 8, the payload is 256 B, and the batch size is 400. The results show that the maximum throughput of RAFTEL and CHAINED-RAFTEL is 10.7 kTPS and 11.1 kTPS, respectively. RAFTEL achieves significantly better performance compared to Damysus and

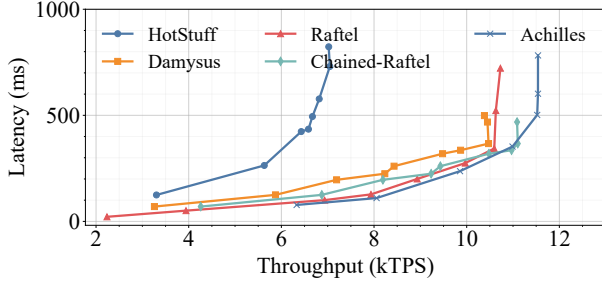


Figure 10. End-to-end Throughput vs. Latency of RAFTEL in LAN.

HotStuff due to its optimal quorum size of $f + 1$ and two-phase commit. HotStuff performs worse than Damysus, with a maximum throughput of 7.0 kTPS, since it requires $3f + 1$ replicas and $2f + 1$ quorum size. Achilles shows the highest throughput with a maximum of 11.5 kTPS because of its minimized commit phase and smallest replica size.

F Discussion

F.1 Dynamic Membership and Reconfiguration

While our design assumes a static configuration, it can be extended to support dynamic membership through reconfiguration [19]. Replica joins and leaves are reflected via configuration updates, with quorum formation always based on the active configuration. Our approach also accommodates changes in TEE availability: replicas can be reclassified between TEE and non-TEE roles (*e.g.*, upon attestation or loss of trust), which updates the corresponding quorum conditions. Upon reconfiguration, the protocol adapts accordingly—enabling TEE-based optimizations (*e.g.*, dual-quorum and TEE-leader acceleration) when sufficient TEE guarantees are available, and otherwise falling back to standard BFT behavior. This ensures safety under dynamic conditions while preserving performance benefits when TEE assumptions hold.

F.2 Extensibility to Other BFT Protocols

Although RAFTEL is instantiated on a HotStuff-style protocol, its key ideas are not specific to chained consensus. Our use of TEE-based non-equivocation to optimize quorum formation and certification can potentially benefit other BFT protocols as well, including Multi-BFT consensus [2, 26, 47, 48, 65] and DAG-based BFT consensus [38, 64]. In particular, many Multi-BFT and DAG-based systems internally instantiate multiple single-leader BFT consensus instances. In principle, RAFTEL can serve as the underlying consensus component for these instances, enabling TEE-aware quorum construction and fast-path execution.

However, fully integrating RAFTEL into these protocols is non-trivial. Their global ordering logic, voting dependencies, and commit rules are often tightly coupled with DAG

structures or cross-instance interactions. Supporting TEE-aware quorum optimizations in these settings would therefore require protocol-specific redesign and new formal safety analysis. We leave such extensions to future work.

G Pseudocode of CHAINED-RAFTEL

We append the pseudocode of CHAINED-RAFTEL in Algorithm 3.

Algorithm 3 The pseudocode of operations for replica i in CHAINED-RAFTEL

(a) Non-trusted code of replica i

```

1:  $pks$  // public keys
2:  $view = 1$  // current view
3:  $qc_{prep}$  // latest prepared certificate
4: blocks // mapping from views to proposed blocks
5:
6: // prepare phase
7: as a leader
8:   if  $qc_{prep}.cview \neq view - 1$  then
9:     // don't have the latest certificate
10:    waits for  $\vec{\phi}$  s.t.  $T\text{-match}(\vec{\phi}, Q_T, \perp, view - 1, nv)$ 
11:       $\vee M\text{-match}(\vec{\phi}, Q_M, \perp, view - 1, nv)$ 
12:     $\phi' :=$  certificate  $\phi \in \vec{\phi}$  with highest  $\phi.V_{Just}$ 
13:  endif
14:   $b := \text{createChain}(\phi', txs)$ 
15:   $blocks[view] := b$ 
16:   $b_0 := blocks[b.just.view]$ 
17:  abort if  $H(blocks[b.just.view]) \neq b.just.hash$ 
18:  if  $isTEE(i)$  then
19:    send  $\phi_{prep} := TEEprepare(b, b_0)$  to all
20:    send  $\phi_{nv} := TEEsign()$  to  $ldr(view + 1)$ 
21:  else
22:    send  $\langle prep, b, H(b), view \rangle_\sigma$  to all
23:    send  $\phi_{nv} := \langle nv, H(b), view \rangle_\sigma$  to replica  $ldr(view + 1)$ 
24:  endif
25: all replicas
26:   waits for  $\langle prep, b, h, view \rangle_\sigma$  from the leader
27:   abort if  $view \neq b.just.view + 1$ 
28:    $b_0 := blocks[b.just.view]$ 
29:   abort if  $H(blocks[b.just.view]) \neq b.just.hash$ 
30:    $b_1 := blocks[b_0.just.view]$ 
31:   abort if  $H(blocks[b_0.just.view]) \neq b_0.just.hash$ 
32:    $b_2 := blocks[b_1.just.view]$ 
33:   abort if  $H(blocks[b_1.just.view]) \neq b_1.just.hash$ 
34:   if  $i \neq ldr(view)$ 
35:      $\phi_{prep} := \langle prep, H(b), view, \perp, \perp \rangle_\sigma$ 
36:     abort if  $\neg(VERIFY(\phi_{prep}) \wedge b > b.just.hash)$ 
37:      $blocks[view] := b$ 
38:     if  $isTEE(i)$  then
39:       send  $\phi' := TEEprepare(b, b_0)$  to replica  $ldr(view + 1)$ 
40:     else
41:       send  $\phi_{nv} := TEEsign()$  to replica  $ldr(view + 1)$ 
42:     endif
43:     send  $\langle prep, b, H(b), view \rangle_\sigma$  to all
44:     send  $\phi_{nv} := \langle nv, H(b), view \rangle_\sigma$  to replica  $ldr(view + 1)$ 
45:   endif
46:   if  $b.parent = H(b_0) \wedge isTEE(ldr(b_0.view))$ 
47:     execute  $b_0$  (and previous block) and reply to the client
48:   endif
49:   if  $b.parent = H(b_0) \wedge b_0.parent = H(b_1)$ 
50:      $\wedge b_1.parent = H(b_2) \wedge \neg isTEE(ldr(b_0.view))$ 
51:     execute  $b_2$  (and previous block) and reply to the client
52:   endif
53:   if  $i \neq ldr(view + 1)$  then  $view++$ 
54:   else
55:     waits for  $\vec{\phi}$  s.t.  $T\text{-match}(\vec{\phi}, Q_T, \perp, h, view, prep)$ 
56:        $\vee M\text{-match}(\vec{\phi}, Q_M, \perp, h, view, prep)$ 
57:      $qc_{prep} = \langle view, h, \vec{\sigma} \rangle; view++$ 
58:   // new-view phase
59:   upon timeout
60:      $(v, ph) := (0, prep); view++$ 
61:     while  $(v, ph) \neq (view, nv)$  do
62:        $(v, ph) := (\phi.view, \phi.phase)$ 
63:       if  $isTEE(i)$  then
64:          $\phi := TEEview();$ 
65:       else
66:          $\phi_v := \langle nv, view, qc_{prep} \rangle_{\sigma_i}$ 
67:       endif
68:     end while
69:     send  $\phi_v$  to  $view$ 's leader

```

(b) TEE code if replica i has TEE

```

69:  $sk, pks$  // private and public key
70:  $(view, phase) = (0, 0)$  // current view and phase
71:  $(prepv, preph) = (0, H(\mathcal{G}))$  // latest prepared block
72:  $(lockv, lockh) = (0, H(\mathcal{G}))$  // latest locked block
73:
74: function  $TEEsing(h, h', v')$ ,  $TEEview()$ 
75:   // Same as in Algorithm 1
76:
77: function  $TEEprepare(b, b_0)$ 
78:    $qc := b.just$ 
79:   if  $\left( \begin{array}{l} VERIFY(qc) \wedge view = qc.cview + 1 \\ \wedge qc.hash = H(b_0) \end{array} \right)$  then
80:     abort if  $\neg(VERIFY(\sigma) \wedge v = view \wedge ph = nv)$ 
81:     abort if  $\neg(H(b) = h \wedge b.h_p = h')$ 
82:     abort if  $\neg(h' = lockh \vee v' > lockv)$ 
83:     if  $b.parent = H(b_0)$  then
84:        $preph := qc.hash; prepv := qc.view$ 
85:     endif
86:     return  $\phi' := TEEsign(H(b), \perp, \perp)$ 
87:   endif
88:
89: function  $TEEstore(\phi_{nv}, \vec{\phi}_n)$ 
90:   // Same as in Algorithm 1

```
